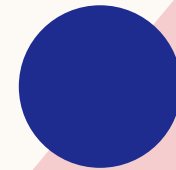


QUERY PROCESSING & OPTIMIZATION

Module 4
BCSE302L – Database
Systems

AGENDA

Query processing
Query Optimization
Algebraic Query Optimization
Heuristic Query Optimization
Join Query Optimization





INTRODUCTION

- DBMS to process, optimize, and execute high-level queries.
- A query expressed in a high-level query language such as SQL, must first be scanned, parsed, and validated
- **Scanner**
 - identifies the query tokens
 - SQL keywords, attribute names, and relation names that appear
- **Parser**
 - checks the query syntax to determine whether it is according to the syntax rules (rules of grammar) of the query language.



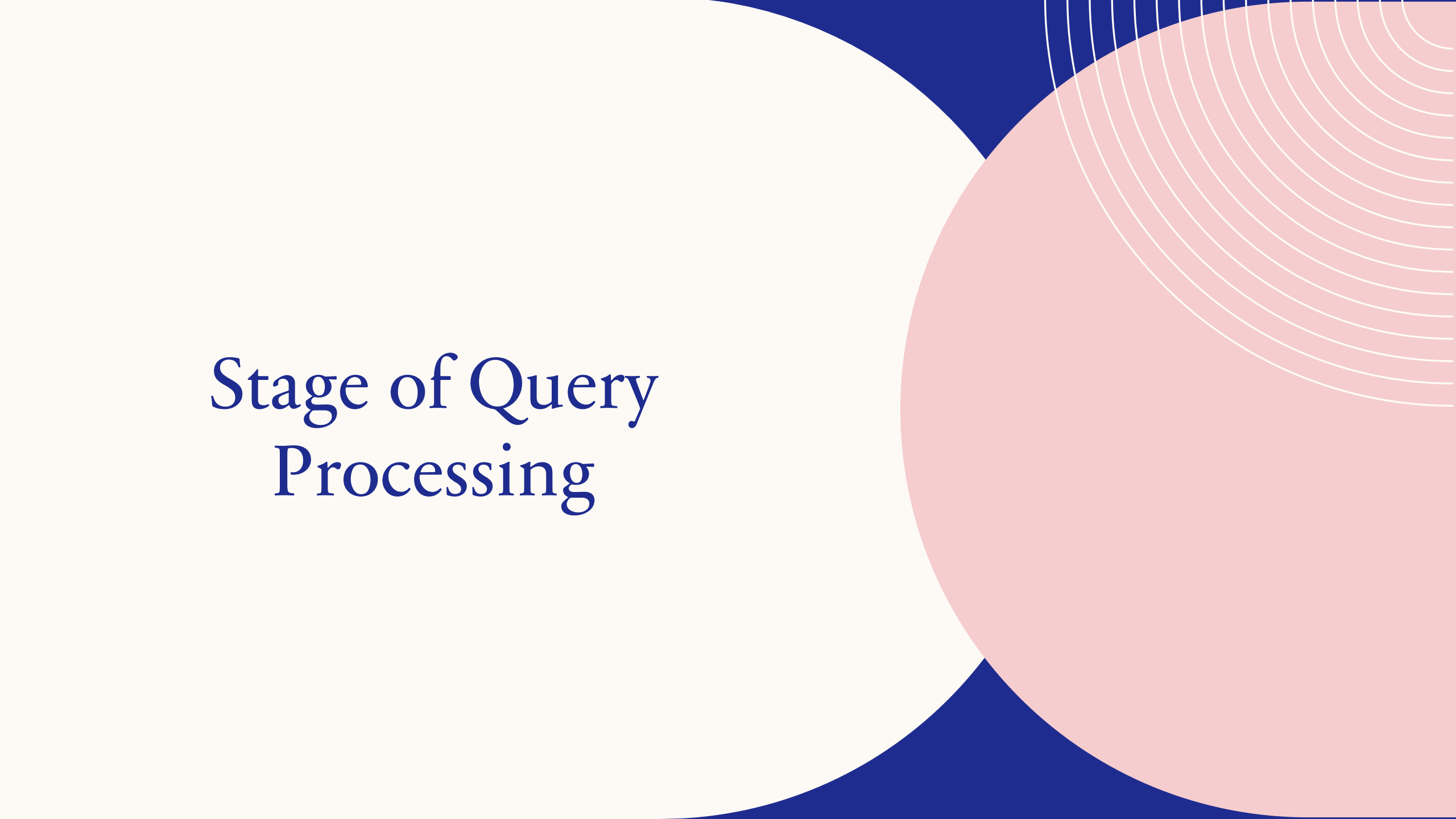
INTRODUCTION

- **Validation**
 - checking that all attribute and relation names are valid and semantically meaningful names in the schema of the particular database being queried
- **Query Tree**
 - It is an internal representation of the query is then created, usually as a tree data structure.
- **Query Graph**
 - Graph data structure to represent the query
- The DBMS must then devise an **execution strategy or query plan** for retrieving the results of the query from the database files.



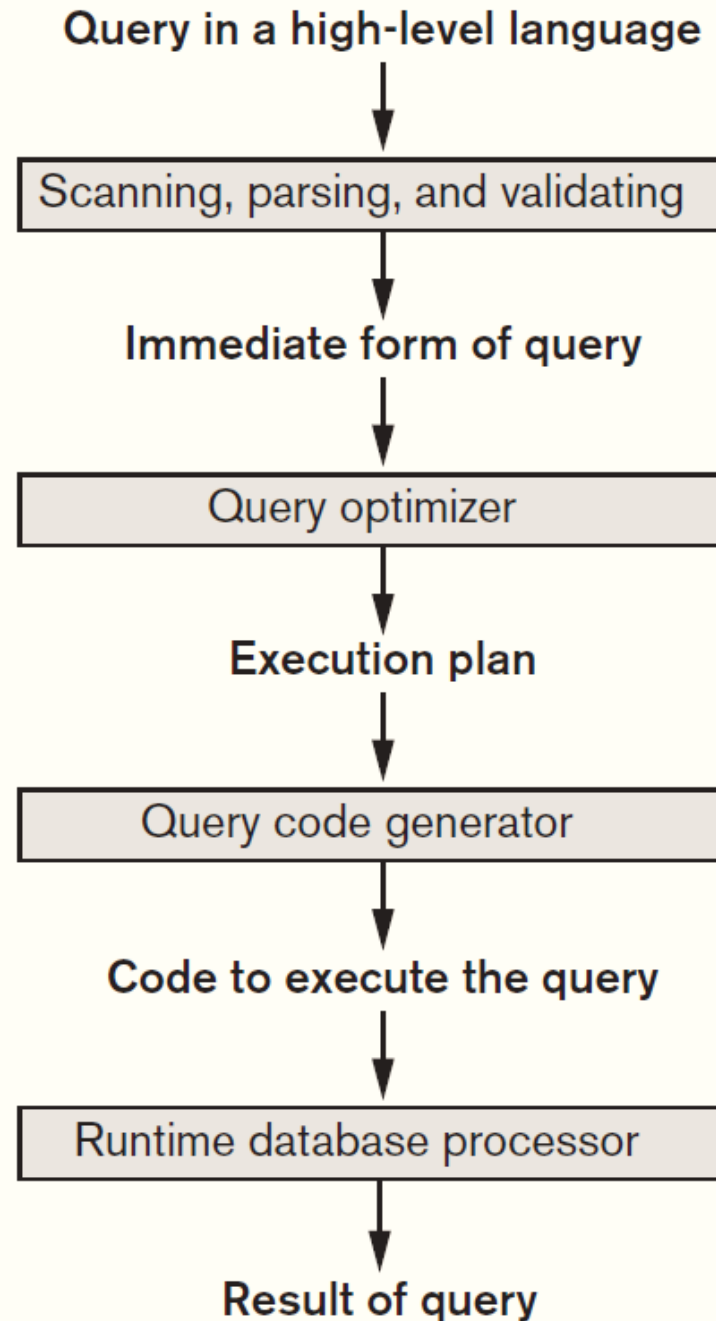
INTRODUCTION

- **Query Optimization**
 - A query typically has many possible execution strategies, and the process of choosing a suitable one for processing a query is known as query optimization.



Stage of Query Processing

Steps when processing a query



Code can be:

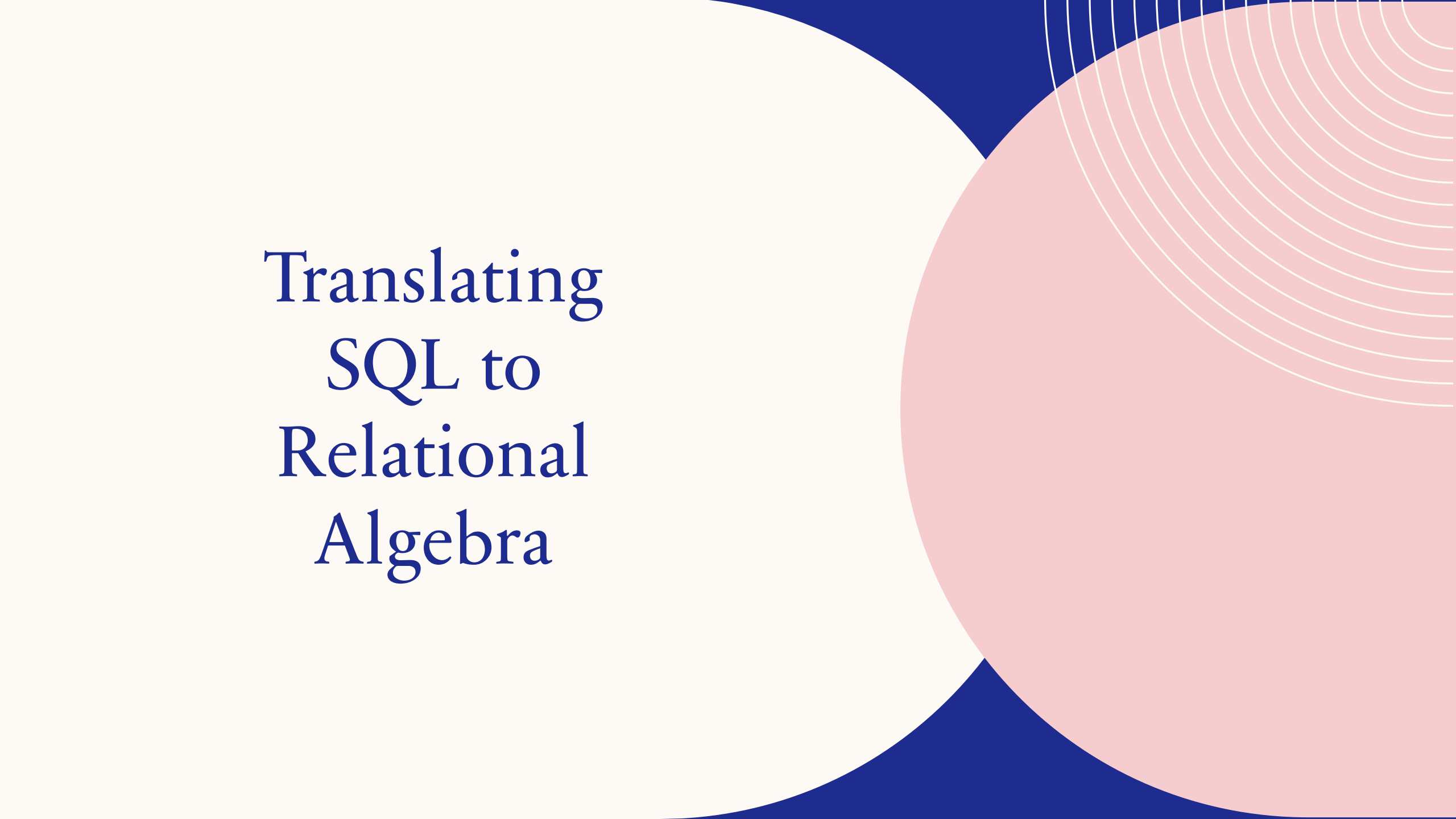
Executed directly (interpreted mode)
Stored and executed later whenever needed (compiled mode)

STAGES OF QUERY PROCESSING

- **Query optimizer** module has the task of producing a good execution plan.
- **Code generator** generates the code to execute that plan.
- **Runtime database processor** has the task of running (executing) the query code, whether in compiled or interpreted mode, to produce the query result.
 - If a runtime error results, an error message is generated by the runtime database processor.

STAGES OF QUERY PROCESSING

- **Two main technique for Query Optimization**
- **Heuristic Rules**
 - Used for ordering the operations in a query execution strategy.
 - A heuristic is a rule that works well in most cases but is not guaranteed to work well in every case.
 - The rules typically reorder the operations in a query tree.
- **Systematic Estimation**
 - Estimating the cost of different execution strategies
 - Choosing the execution plan with the lowest cost estimate.

The background features a large, light pink circle on the right side, which overlaps with a dark blue circle on the left. The dark blue circle is partially visible, showing its upper and lower arcs. The pink circle contains several thin, white, concentric circular lines that are more densely packed towards the top right edge.

Translating SQL to Relational Algebra

TRANSLATING SQL TO RELATIONAL ALGEBRA

- SQL query is first translated into an equivalent extended relational algebra expression represented as a query tree data structure that is then optimized.
- SQL queries are decomposed into query blocks, which form the basic units that can be translated into the algebraic operators and optimized.
- A **query block** contains a single SELECT-FROM-WHERE expression, as well as GROUP BY and HAVING clauses if these are part of the block.
- Nested Query are identified as separate query blocks.

EMPLOYEE

Fname	Minit	Lname	<u>Ssn</u>	Bdate	Address	Sex	Salary	Super_ssn	Dno
-------	-------	-------	------------	-------	---------	-----	--------	-----------	-----

DEPARTMENT

Dname	<u>Dnumber</u>	Mgr_ssn	Mgr_start_date
-------	----------------	---------	----------------

DEPT_LOCATIONS

<u>Dnumber</u>	<u>Dlocation</u>
----------------	------------------

PROJECT

Pname	<u>Pnumber</u>	Plocation	Dnum
-------	----------------	-----------	------

WORKS_ON

<u>Essn</u>	<u>Pno</u>	Hours
-------------	------------	-------

DEPENDENT

<u>Essn</u>	<u>Dependent_name</u>	Sex	Bdate	Relationship
-------------	-----------------------	-----	-------	--------------

TRANSLATING SQL TO RELATIONAL ALGEBRA

- Find the first and last name of the employee who earn a salary that is greater than the highest salary in department 5.
- Consider the following SQL Query on the EMPLOYEE relation

```
SELECT Lname, Fname FROM EMPLOYEE WHERE Salary >  
(SELECT MAX (Salary) FROM EMPLOYEE WHERE Dno=5 );
```

- Query includes a nested subquery and decompose into two blocks.

TRANSLATING SQL TO RELATIONAL ALGEBRA

SELECT Lname, Fname **FROM** EMPLOYEE **WHERE** Salary >
(**SELECT** MAX (Salary) **FROM** EMPLOYEE **WHERE** Dno=5);

- Inner block:

SELECT MAX (Salary) **FROM** EMPLOYEE **WHERE** Dno=5

- Outer block:

SELECT Lname, Fname **FROM** EMPLOYEE **WHERE** Salary > C

- Where C represents the inner block.

TRANSLATING SQL TO RELATIONAL ALGEBRA

- Translating Inner block into Relational Algebra expression:

SELECT MAX (Salary) FROM EMPLOYEE WHERE Dno=5

$\rho_{MAX(salary)}(\sigma_{dno=5}(EMPLOYEE))$

- Outer block:

SELECT Lname, Fname FROM EMPLOYEE WHERE Salary > C

$\Pi_{Lname, Fname}(\sigma_{salary > c}(EMPLOYEE))$

- Query optimizer would then choose an execution plan for each query block.
- This is an example of **Nested Query without correlation**.

External Sorting

The background features a large, light cream-colored circle on the left. To its right is a large, light pink circle. The top and bottom edges of the image are filled with a solid dark blue color. In the upper right corner, within the pink circle, there are several thin, white, concentric curved lines that fan out from the top edge.

ALGORITHM FOR EXTERNAL SORTING

- Sorting is one of the primary algorithms used in query processing.
- SQL query specifies an ORDER BY-clause, the query result must be sorted.
- **Sorting is also a key component in**
 - Sort-merge algorithms used for JOIN
 - UNION and INTERSECTION
 - Duplicate elimination algorithms for the PROJECT operation (using DISTINCT in the SELECT clause)

ALGORITHM FOR EXTERNAL SORTING

- **External Sorting:**
- Suitable for large files of records stored on disk that do not fit in main memory
- **Sort-merge strategy**
- Starts by sorting small subfiles called **runs** of the main file and then **merges** the sorted runs, creating larger sorted subfiles that are merged in turn.
- It consist of 2 phases – **sorting phase** and **merging phase**
- Requires Buffer space (part of DBMS Cache) in main memory, divided into individual buffers of size *equal to size of one disk block*.

SORT MERGE STRATEGY

- **Sorting Phase**
- Runs (portions or pieces) of the file that can fit in buffer space are read into main memory, sorted using an internal sorting algorithm, and written back to disk as temporary sorted subfiles (or runs).
- The size of each run depends on
 - Number of initial runs (n_R)
 - Number of file blocks (b)
 - Buffer space (n_B).

SORT MERGE STRATEGY

- **Sorting Phase**
- Example : if the number of available main memory buffers $n_B = 5$ disk blocks and the size of the file $b = 1024$ disk blocks, what is size of initial run for given disk blocks?
- Size of initial run, $n_R = \lceil (b/n_B) \rceil = \lceil (1024/5) \rceil = 204 + 1 = 205$ (except the last run which will have only 4 blocks).
- After the sorting phase, 205 sorted runs (or 205 sorted subfiles of the original file) are stored as temporary subfiles on disk.

SORT MERGE STRATEGY

- **Merging Phase**
- Sorted runs are merged during one or more merge passes.
- Each merge pass can have one or more merge steps.
- The degree of merging (d_M) is the number of sorted subfiles that can be merged in each merge step.
- During merging, one buffer block is needed to hold one disk block from each of the sorted subfiles being merged, and one additional buffer is needed for containing one disk block of the merge result, which will produce a larger sorted file that is the result of merging several smaller sorted subfiles

SORT MERGE STRATEGY

- **Merging Phase**
- d_M is the smaller of $(n_B - 1)$ and n_R , and the number of merge passes is $\lceil \log_{d_M}(n_R) \rceil$.
- Example: $n_B = 5$, $d_M = 4$ (four-way merging), so the 205 initial sorted runs would be merged 4 at a time in each step into 52 larger sorted subfiles at the end of the first merge pass.
- These 52 sorted files are then merged 4 at a time into 13 sorted files, which are then merged into 4 sorted files, and then finally into 1 fully sorted file, which means that **four passes are needed**.

SORT MERGE STRATEGY

```
set       $i \leftarrow 1$ ;  
          $j \leftarrow b$ ;           {size of the file in blocks}  
          $k \leftarrow n_B$ ;       {size of buffer in blocks}  
          $m \leftarrow \lceil (j/k) \rceil$ ;  
  
{Sorting Phase}  
while ( $i \leq m$ )  
do {  
    read next  $k$  blocks of the file into the buffer or if there are less than  $k$  blocks  
    remaining, then read in the remaining blocks;  
    sort the records in the buffer and write as a temporary subfile;  
     $i \leftarrow i + 1$ ;  
}
```

SORT MERGE STRATEGY

```
{Merging Phase: merge subfiles until only 1 remains}
set       $i \leftarrow 1$ ;
           $p \leftarrow \lceil \log_{k-1} m \rceil$   { $p$  is the number of passes for the merging phase}
           $j \leftarrow m$ ;
while ( $i \leq p$ )
do {
     $n \leftarrow 1$ ;
     $q \leftarrow \lceil j/(k-1) \rceil$ ; {number of subfiles to write in this pass}
    while ( $n \leq q$ )
    do {
        read next  $k-1$  subfiles or remaining subfiles (from previous pass)
        one block at a time;
        merge and write as new subfile one block at a time;
         $n \leftarrow n + 1$ ;
    }
     $j \leftarrow q$ ;
     $i \leftarrow i + 1$ ;
}
```


SORT MERGE STRATEGY

- **Merging Phase**

- The performance of the sort-merge algorithm can be measured in the number of disk block reads and writes (between the disk and main memory) before the sorting of the whole file is completed.
- The following formula approximates this **cost**:

$$(2 * b) + (2 * b * (\log_{dM}(n_R)))$$

- **Number of merge passes** is $(\log_{dM}(n_R))$,
- **Total merge cost** of $(2 * b * (\log_{dM}(n_R)))$.

The background features a large white circle on the left and a large pink circle on the right, both partially overlapping a dark blue background. The pink circle contains several thin, white, concentric circular lines.

Algorithm for Select Operation

ALGORITHMS FOR SELECT OPERATIONS

- Basically a search operation to locate the records in a disk file that satisfy a certain condition.
- Some of the search algorithms depend on the file having specific access paths, and they may apply only to certain types of selection conditions.
 - **Search methods for Simple Selection**
 - **Search methods for Complex Selection**
 - **Selectivity of a condition**
 - **Disjunctive Selection conditions**

SEARCH METHOD FOR SIMPLE SELECTION

- File scan – scan the records of a file to search for and retrieve records that satisfy a selection condition.
- **Index scan** – search algorithm use of an index.
- **To implement a select operation using following search methods**
 - **S1 – Linear Search**
 - **S2 – Binary Search**
 - **S3a – using a primary index**
 - **S3b – using a hash key**
 - **S4 – using a primary index to retrieve multiple records**
 - **S5 – using clustering index to retrieve multiple records**
 - **S6 – using a secondary (B⁺ tree) index on an equality comparison**

SEARCH METHOD FOR SIMPLE SELECTION

- **S1 – Linear Search**
 - Retrieve every record in the file, and test whether its attribute values satisfy the selection condition.
 - The records are grouped into disk blocks, each disk block is read into a main memory buffer, and then a search through the records within the disk block is conducted.

SEARCH METHOD FOR SIMPLE SELECTION

- **S2 – Binary Search**

- If the selection condition involves an equality comparison on a key attribute on which the file is ordered.

$$\sigma_{ssn="12345678"} (EMPLOYEE)$$

- The attribute ssn must be ordered in EMPLOYEE.

- **S3 – using a primary index**

- If the selection condition involves an equality comparison on a key attribute with a primary index.

$$\sigma_{ssn="12345678"} (EMPLOYEE)$$

- Query use the primary index and retrieves a single record.

SEARCH METHOD FOR SIMPLE SELECTION

- **S3b – using a hash key**

- If the selection condition involves an equality comparison on a key attribute with a hash key

$$\sigma_{ssn="12345678"} (EMPLOYEE)$$

- Query use the hash key to retrieve and retrieves a single record.

- **S4 – using a primary index to retrieve multiple records**

- if the comparison condition is >, >=, <, or <= on a key field with a primary index.

$$\sigma_{dno>5} (DEPARTMENT)$$

- Query use index to find the record satisfying equality condition, then retrieve all subsequent records in ordered file.

SEARCH METHOD FOR SIMPLE SELECTION

- **S5 – using clustering index to retrieve multiple records**
 - If the selection condition involves an equality comparison on a nonkey attribute with a clustering index
$$\sigma_{dno=5} (EMPLOYEE)$$
 - Query use index to retrieve all the record satisfying the condition.
- **S6 – using a secondary (B+ tree) index on an equality comparison**
 - Search method can be used to retrieve a single record if the indexing field is a **key** (has unique values) or to retrieve multiple records if the indexing field is **not a key**.

SEARCH METHOD FOR SIMPLE SELECTION

- Linear search Method - S1
- Binary Search Method - S2
- Index Search Method - S3a, S4, S5, S6
- Range based Method – S4 and S6
- If the selection condition involves an equality comparison on a nonkey attribute with a clustering index

$\sigma_{dno=5}$ (*EMPLOYEE*)

- Query use index to retrieve all the record satisfying the condition.

SEARCH METHOD FOR COMPLEX SELECTION

- If a condition of a SELECT operation is **a conjunctive condition**—that is, if it is made up of several simple conditions connected with the AND logical connective.

$\sigma_{dno=5 \wedge salary>3000 \wedge sex="F"} (EMPLOYEE)$

- To implement a select operation using following methods
 - S7 – Conjunctive selection using an individual index
 - S8 – Conjunctive selection using a composite index
 - S9 – Conjunctive selection by intersection of record pointers

SEARCH METHOD FOR COMPLEX SELECTION

- **S7 – Conjunctive selection using an individual index**
- Single attribute involved in any **simple condition in the conjunctive select condition.**
- It has an access path that permits the use of one of the methods S2 to S6.
- Using that condition to retrieve the records and then check whether each retrieved record satisfies the remaining simple conditions in the conjunctive select

SEARCH METHOD FOR COMPLEX SELECTION

- **S8 – Conjunctive selection using a composite index**
- Two or more attribute involved in equality condition in the conjunctive select condition.
- A composite index exists on the combined fields.

$\sigma_{essn="12345" \wedge pno=10} (WORKS_ON)$

- If an index is created on the composite key (essn, pno) of WORKS_ON, then we can use the index directly.

SEARCH METHOD FOR COMPLEX SELECTION

- **S9 – Conjunctive selection by intersection of record pointers**
- If secondary indexes are available on more than one of the fields involved in simple conditions in the conjunctive select condition,
- And if the indexes include record pointers (rather than block pointers), then each index can be used to retrieve the set of record pointers that satisfy the individual condition.
- The intersection of these sets of record pointers gives the record pointers that satisfy the conjunctive select condition, which are then used to retrieve those records directly.

SELECTIVITY OF A CONDITION

- Optimizer is choosing between multiple simple conditions in a conjunctive select condition, it typically considers the selectivity of each condition
- **Selectivity (sl)** is defined as the ratio of the number of records (tuples) that satisfy the condition to the total number of records (tuples) in the file (relation), and thus is a number between zero and one.
- **Zero selectivity** means none of the records in the file satisfies the selection condition, and a selectivity of one means that all the records in the file satisfy the condition.

SELECTIVITY OF A CONDITION

- Selectivity will not be either of these two extremes, but will be a fraction that estimates the percentage of file records that will be retrieved.
- Estimates of selectivities for an equality condition on **key attribute**:

$$s = 1 / |r(R)|$$

where $|r(R)|$ is the number of tuples in relation $r(R)$.

- Estimates of selectivities for an equality condition on **nonkey attribute**:

$$s = (|r(R)| / i) / |r(R)| \text{ or } s = 1 / i,$$

where $|r(R)|$ is the number of tuples in relation $r(R)$ and $|r(R)|/i$ records will satisfy an equality condition on this attribute.

DISJUNCTIVE SELECTION CONDITIONS

- Compared to a conjunctive selection condition, a disjunctive condition (where simple conditions are connected by the OR logical connective rather than by AND) is much harder to process and optimize.

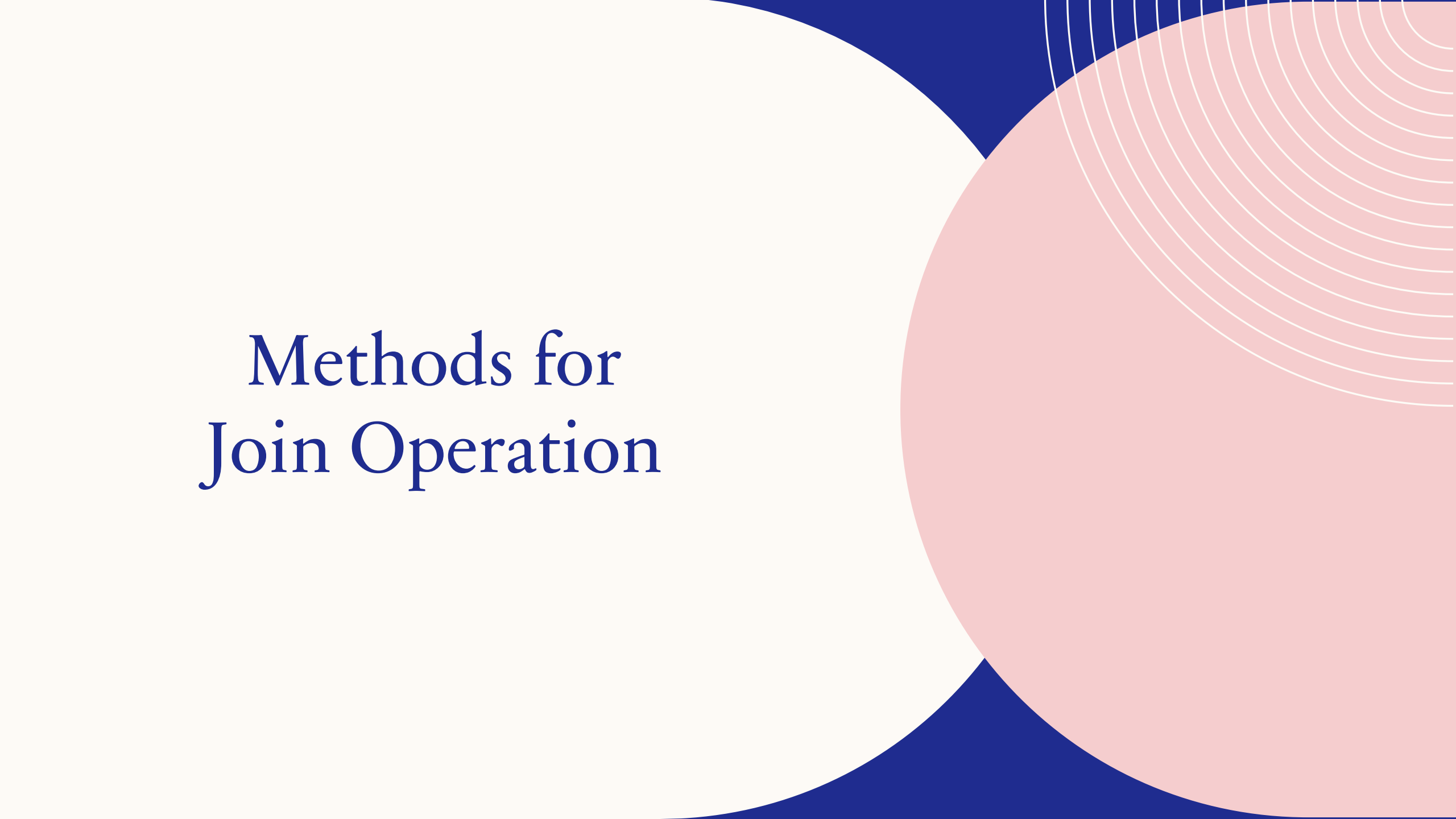
$\sigma_{dno=5 \wedge salary>3000 \wedge sex="F"} (EMPLOYEE)$

- Optimization can be done, because the records satisfying the disjunctive condition are the union of the records satisfying the individual conditions.

DISJUNCTIVE SELECTION CONDITIONS

$\sigma_{dno=5 \wedge salary>3000 \wedge sex="F"} (EMPLOYEE)$

- Hence, if any one of the conditions does not have an access path, we are compelled to use the brute force, linear search approach.
- Only if an access path exists on every simple condition in the disjunction can we optimize the selection by retrieving the records satisfying each condition or their record ids and then applying the union operation to eliminate duplicates.

The background features a large white circle on the left and a large pink circle on the right, both overlapping a dark blue background. The pink circle contains several thin, white, concentric circular lines.

Methods for Join Operation

IMPLEMENTING THE JOIN OPERATION

- The JOIN operation is one of the most time-consuming operations in query processing.
- The term join refers to an EQUIJOIN (or NATURAL JOIN).
- A **two-way join**, which is a join on two files.
- **Multiway Joins** involving more than two files.
- The number of possible ways to execute multiway joins grows very rapidly.

$$R \bowtie_{A=B} S$$

$$EMPLOYEE \bowtie_{dno=dnumber} DEPARTMENT$$

$$DEPARTMENT \bowtie_{mgr_ssn=ssn} EMPLOYEE$$

IMPLEMENTING THE JOIN OPERATION

- **Method for implementing Joins**
 - J1 – Nested-loop join (or nested-block join)
 - J2 – Single-loop join (using an access structure to retrieve the matching records)
 - J3 – Sort-merge join
 - J4 – Partition-hash join

IMPLEMENTING THE JOIN OPERATION

- **J1 – Nested-loop join (or nested-block join)**
 - Default Algorithm, does not require any special access path on either file in the join.
 - For each record t in R (outer loop), retrieve every record s from S (inner loop) and test whether the two records satisfy the join condition $t[A] = s[B]$
- **J2 – Single-loop join**
 - If an index (or hash key) exists for one of the two join attributes say, attribute B of file S —retrieve each record t in R (loop over file R), and then use the access structure (such as an index or a hash key) to retrieve directly all matching records s from S that satisfy $s[B] = t[A]$.

IMPLEMENTING THE JOIN OPERATION

- **J3 – Sort-merge join**

- If the records of **R** and **S** are physically sorted (ordered) by value of the join attributes **A** and **B**, respectively, we can implement the join in the most efficient way possible.
- Both files are scanned concurrently in order of the join attributes, matching the records that have the same values for **A** and **B**.
- If the files are not sorted, they may be sorted first by using external sorting.
- Pairs of file blocks are copied into memory buffers in order and the records of each file are scanned only once each for matching with the other file unless both **A** and **B** are nonkey attributes, in which case the method needs to be modified slightly.

IMPLEMENTING THE JOIN OPERATION

- **J4 – Partition-hash join**

- The records of files **R** and **S** are partitioned into smaller files.
- The partitioning of each file is done using the same hashing function **h** on the join attribute **A** of **R** (for partitioning file **R**) and **B** of **S** (for partitioning file **S**).
- First, a single pass through the file with fewer records (say, **R**) hashes its records to the various partitions of **R**; this is called the **partitioning phase**, since the records of **R** are partitioned into the hash buckets.
- The smaller file can fit entirely in main memory after it is partitioned, so that the partitioned subfiles of **R** are all kept in main memory.
- The collection of records with the same value of **h(A)** are placed in the same partition, which is a **hash bucket** in a hash table in main memory.

IMPLEMENTING THE JOIN OPERATION

- **J4 – Partition-hash join**
- In the second phase, called the **probing phase**, a single pass through the other file (**S**) then hashes each of its records using the same hash function **$h(B)$** to probe the appropriate bucket, and that record is combined with all matching records from **R** in that bucket.
- Partition-hash join assumes that the smaller of the two files fits entirely into memory buckets after the first phase.

**Implementing JOIN,
PROJECT, UNION,
INTERSECTION, and
SET DIFFERENCE by
using sort-merge, where
R has n tuples and S has
m tuples.**

**Algorithm for
Implementation of**

$$T \leftarrow R \bowtie_{A=B} S$$

```

sort the tuples in R on attribute A;
sort the tuples in S on attribute B;
set  $i \leftarrow 1, j \leftarrow 1$ ;
while ( $i \leq n$ ) and ( $j \leq m$ )
do {
    if  $R(i)[A] > S(j)[B]$ 
        then set  $j \leftarrow j + 1$ 
    elseif  $R(i)[A] < S(j)[B]$ 
        then set  $i \leftarrow i + 1$ 
    else {
        (*  $R(i)[A] = S(j)[B]$ , so we output a matched tuple *)
        output the combined tuple  $\langle R(i), S(j) \rangle$  to T;

        (* output other tuples that match R(i), if any *)
        set  $l \leftarrow j + 1$ ;
        while ( $l \leq m$ ) and ( $R(i)[A] = S(l)[B]$ )
        do {
            output the combined tuple  $\langle R(i), S(l) \rangle$  to T;
            set  $l \leftarrow l + 1$ 
        }

        (* output other tuples that match S(j), if any *)
        set  $k \leftarrow i + 1$ ;
        while ( $k \leq n$ ) and ( $R(k)[A] = S(j)[B]$ )
        do {
            output the combined tuple  $\langle R(k), S(j) \rangle$  to T;
            set  $k \leftarrow k + 1$ 
        }
        set  $i \leftarrow k, j \leftarrow l$ 
    }
}

```

(* assume R has n tuples (records) *)
 (* assume S has m tuples (records) *)

PERFORMANCE OF NESTED LOOP JOIN

- **J1 – Nested-Loop join – choosing file for inner and outer loop**
 - Consider 7 blocks of buffers available in main memory ($n_B = 7 \text{ block}$).
 - DEPARTMENT file 50 records stored in 10 disk blocks ($r_D = 50, b_D = 10$)
 - EMPLOYEE 6000 records store in 2000 disk blocks ($r_E = 6000, b_E = 2000$)

EMPLOYEE ⋈_{dno=dnumber} *DEPARTMENT*

Note: To read as many blocks as possible at a time into main memory from the file. The algorithm read one block at a time for inner loop file – use for search in outer loop for matching. This reduce the total block access.

PERFORMANCE OF NESTED LOOP JOIN

- **J1 – Nested-Loop join**

EMPLOYEE ⋈_{dno=dnumber} *DEPARTMENT*

- $n_B = 7$, DEPARTMENT - $r_D = 50$, $b_D = 10$, EMPLOYEE - $r_E = 6000$, $b_E = 2000$
- In nested loop join, it makes difference which file is chosen for outer loop and inner loop.
- EMPLOYEE is used for outer loop (read one block at time), DEPARTMENT is used as inner loop (read $n_B - 2$ block for each time).

PERFORMANCE OF NESTED LOOP JOIN

- $n_B = 7, r_D = 50, b_D = 10, r_E = 6000, b_E = 2000$
- Total number of blocks read for outer loop = b_E
- Number of time outer loop file loaded into main memory = $\lceil b_E / (n_B - 2) \rceil$
- Total number of blocks read for inner loop file = $b_D * \lceil b_E / (n_B - 2) \rceil$
- Total number of blocks read for accesses = $b_E + (b_D * \lceil b_E / (n_B - 2) \rceil) = 2000 + (10 * \lceil (2000/5) \rceil) = 6000$ block access
- DEPARTMENT is outer loop and EMPLOYEE is inner loop
- Total number of blocks read for accesses = $b_D + (b_E * \lceil b_D / (n_B - 2) \rceil) = 10 + (2000 * \lceil (10/5) \rceil) = 4010$ block access

PERFORMANCE OF SINGLE LOOP JOIN

- **J2 – Single-Loop join – choosing a selection factor for join**
 - Consider DEPARTMENT file 50 records stored in 10 disk blocks ($r_D = 50, b_D = 10$), EMPLOYEE 6000 records store in 2000 disk blocks ($r_E = 6000, b_E = 2000$)

DEPARTMENT ⋈_{mgr_ssn = ssn} *EMPLOYEE*

Note: DEPARTMENT record (there are 50 such records in our example) will be joined with a single EMPLOYEE record, but many EMPLOYEE records (the 5,950 of them that do not manage a department) will not be joined with any record from DEPARTMENT.

PERFORMANCE OF SINGLE LOOP JOIN

- **J2 – Single-Loop join – choosing a selection factor for join**
- DEPARTMENT file 50 records stored in 10 disk blocks ($r_D = 50$, $b_D = 10$) EMPLOYEE 6000 records store in 2000 disk blocks ($r_E = 6000$, $b_E = 2000$)
- Consider secondary index available on EMPLOYEE(ssn) and DEPARTMENT(mgr_ssn) number of index level $x_{ssn} = 4$ and $x_{mgr_ssn} = 2$.

$DEPARTMENT \bowtie_{mgr_ssn = ssn} EMPLOYEE$

- Each EMPLOYEE record and then uses the index on Mgr_ssn of DEPARTMENT to find a matching DEPARTMENT

PERFORMANCE OF SINGLE LOOP JOIN

- $r_D = 50, b_D = 10, r_E = 6000, b_E = 2000, x_{ssn} = 4, x_{mgr_ssn} = 2.$

$DEPARTMENT \bowtie_{mgr_ssn = ssn} EMPLOYEE$

- Each EMPLOYEE record and then uses the **index on Mgr_ssn** of DEPARTMENT to find a matching DEPARTMENT
- Total number of block access = $b_E + (r_E * (x_{mgr_ssn} + 1)) = 2000 + (6000 * 3) = 20000$ blocks.
- Each DEPARTMENT record and then uses the **index on ssn** of EMPLOYEE to find a matching EMPLOYEE
- Total number of block access = $b_D + (r_D * (x_{ssn} + 1)) = 10 + (50 * 5) = 260$ blocks.

PERFORMANCE OF SORTED MERGE JOIN

- **J3 – Sorted-merge join**

- Consider DEPARTMENT file 50 records stored in 10 disk blocks ($r_D = 50, b_D = 10$), EMPLOYEE 6000 records store in 2000 disk blocks ($r_E = 6000, b_E = 2000$)

$EMPLOYEE \bowtie_{dno=dnumber} DEPARTMENT$
 $DEPARTMENT \bowtie_{mgr_ssn=ssn} EMPLOYEE$

Note: Only a single pass is made through each file. Hence, the number of blocks accessed is equal to the sum of the numbers of blocks in both files.

- Total number of block access = $b_E + b_D = 2000 + 10 = 2010$ block access.

PERFORMANCE OF PARTITION HASH JOIN

- J4 – Partition-Hash join
- In the partitioning phase, R is partitioned into the M partitions $R_1, R_2, \dots, R_M$, and S into the M partitions $S_1, S_2, \dots, S_M$. The property of each pair of corresponding partitions R_i, S_i with respect to the join operation is that records in R_i only need to be joined with records in S_i , and vice versa.
- Total number of buffer block needed = $M + 1$ blocks.

PERFORMANCE OF PARTITION HASH JOIN

- **J4 – Partition-Hash join**
- In the second phase, called the joining or probing phase, **M iterations are needed**. During iteration **i**, two corresponding partitions **R_i** and **S_i** are joined.
- The minimum number of buffers needed for iteration **i** is the number of blocks in the smaller of the two partitions, say **R_i** , plus two additional buffers.
- Total number of buffer block needed = **$R_i + 2$** blocks.
- Cost of partition hash join = **$3 * (b_R + b_S) + b_{RES}$** blocks.

ALGORITHM FOR PROJECT AND SET OPERATIONS

- **Project**
- **PROJECT** operation $\Pi_{\langle \text{attributes list} \rangle}(R)$ is straightforward to implement
 - If $\langle \text{attribute list} \rangle$ **includes a key** of relation **R**, because in this case the result of the operation will have the same number of tuples as **R**, but with only the values for the attributes in $\langle \text{attribute list} \rangle$ in each tuple.
 - If $\langle \text{attribute list} \rangle$ **does not include a key** of **R**, **duplicate tuples must be eliminated**. This can be done by sorting the result of the operation and then eliminating duplicate tuples, which appear consecutively after sorting.
 - Hashing can be used to eliminate duplicates.

ALGORITHM FOR PROJECT AND SET OPERATIONS

- **Set Operations**
- CARTESIAN PRODUCT operation $R \times S$ is quite expensive because its result includes a record for each combination of records from R and S .
- Each record in the result includes all attributes of R and S .
- If R has n records and j attributes, and S has m records and k attributes, the result relation for $R \times S$ will have $n * m$ records and each record will have $j + k$ attributes.
- Expensive and we avoid by using join.

- Implementing JOIN, PROJECT, UNION, INTERSECTION, and SET DIFFERENCE by using sort-merge, where R has n tuples and S has m tuples. Algorithm for Implementation of $T \leftarrow \Pi_{\langle attributes \rangle}(R)$

```
create a tuple  $t[\langle \text{Attribute list} \rangle]$  in  $T'$  for each tuple  $t$  in R;  
  (*  $T'$  contains the projection results before duplicate elimination *)  
if  $\langle \text{Attribute list} \rangle$  includes a key of  $R$   
  then  $T \leftarrow T'$   
else { sort the tuples in  $T'$ ;  
      set  $i \leftarrow 1, j \leftarrow 2$ ;  
      while  $i \leq n$   
      do { output the tuple  $T'[i]$  to  $T$ ;  
          while  $T'[i] = T'[j]$  and  $j \leq n$  do  $j \leftarrow j + 1$ ;          (* eliminate duplicates *)  
           $i \leftarrow j; j \leftarrow i + 1$   
        }  
      }  
}
```

(* T contains the projection result after duplicate elimination *)

(continues)

ALGORITHM FOR PROJECT AND SET OPERATIONS

- **Set Operations**
- **Type-compatible** (or **union-compatible**) relations, which have the same number of attributes and the same attribute domains.
- Implemented by using variations of the **sort-merge technique**: the two relations are sorted on the same attributes, and, after sorting, a single scan through each relation is sufficient to produce the result.
- **UNION** operation, $R \cup S$, by scanning and merging both sorted files concurrently, and whenever the same tuple exists in both relations, only one is kept in the merged result.
- **INTERSECTION** operation, $R \cap S$, we keep in the merged result only those tuples that appear in both sorted relations.

ALGORITHM FOR PROJECT AND SET OPERATIONS

- **Set Operations**
- **Hashing** used to implement set operations. One table is first scanned and then partitioned into an in-memory hash table with buckets, and the records in the other table are then scanned one at a time and used to probe the appropriate partition.
- $R \cup S$, first hash (partition) the records of R ; then, hash (probe) the records of S , but do not insert duplicate records in the buckets.

- Implementing JOIN, PROJECT, UNION, INTERSECTION, and SET DIFFERENCE by using sort-merge, where R has n tuples and S has m tuples. Algorithm for Implementation of $T \leftarrow R \cup S$

```
sort the tuples in  $R$  and  $S$  using the same unique sort attributes;  
set  $i \leftarrow 1, j \leftarrow 1$ ;  
while  $(i \leq n)$  and  $(j \leq m)$   
do { if  $R(i) > S(j)$   
      then { output  $S(j)$  to  $T$ ;  
            set  $j \leftarrow j + 1$   
          }  
      elseif  $R(i) < S(j)$   
      then { output  $R(i)$  to  $T$ ;  
            set  $i \leftarrow i + 1$   
          }  
      else set  $j \leftarrow j + 1$  (*  $R(i) = S(j)$ , so we skip one of the duplicate tuples *)  
}  
if  $(i \leq n)$  then add tuples  $R(i)$  to  $R(n)$  to  $T$ ;  
if  $(j \leq m)$  then add tuples  $S(j)$  to  $S(m)$  to  $T$ ;
```


- Implementing JOIN, PROJECT, UNION, INTERSECTION, and SET DIFFERENCE by using sort-merge, where R has n tuples and S has m tuples. Algorithm for Implementation of $T \leftarrow R \cap S$

```
sort the tuples in  $R$  and  $S$  using the same unique sort attributes;  
set  $i \leftarrow 1, j \leftarrow 1$ ;  
while ( $i \leq n$ ) and ( $j \leq m$ )  
do {  if  $R(i) > S(j)$   
      then set  $j \leftarrow j + 1$   
      elseif  $R(i) < S(j)$   
      then set  $i \leftarrow i + 1$   
      else {  output  $R(j)$  to  $T$ ;          (*  $R(i)=S(j)$ , so we output the tuple *)  
            set  $i \leftarrow i + 1, j \leftarrow j + 1$   
      }  
}  
}
```

- Implementing JOIN, PROJECT, UNION, INTERSECTION, and SET DIFFERENCE by using sort-merge, where R has n tuples and S has m tuples. Algorithm for Implementation of $T \leftarrow R - S$

sort the tuples in R and S using the same unique sort attributes;

set $i \leftarrow 1, j \leftarrow 1$;

while $(i \leq n)$ and $(j \leq m)$

do { if $R(i) > S(j)$

 then set $j \leftarrow j + 1$

elseif $R(i) < S(j)$

 then { output $R(i)$ to T ;

 set $i \leftarrow i + 1$

 }

else set $i \leftarrow i + 1, j \leftarrow j + 1$

}

(* $R(i)$ has no matching $S(j)$, so output $R(i)$ *)

if $(i \leq n)$ then add tuples $R(i)$ to $R(n)$ to T ;

IMPLEMENTING AGGREGATE OPERATIONS

- **Aggregate Operation – MAX, MIN, COUNT, AVERAGE, SUM**
- Applied to an entire table, can be computed by a table scan or by using an appropriate index, if available.
- The **index also used for the AVERAGE and SUM** aggregate functions, but only if it is a dense index.
- For a nondense index, the actual number of records associated with each index value must be used for a correct computation. This can be done if the *number of records associated with each value in the index* is stored in each index entry.

IMPLEMENTING AGGREGATE OPERATIONS

- **Aggregate Operation – MAX, MIN, COUNT, AVERAGE, SUM**
- COUNT aggregate function, the number of values can be also computed from the index in a similar manner.
- If a COUNT(*) function is applied to a whole relation, the number of records currently in each relation are typically stored in the catalog, and so the result can be retrieved directly from the catalog.
- GROUP BY clause is used in a query, the aggregate operator must be applied separately to each group of tuples as partitioned by the grouping attribute. Hence, the table must first be partitioned into subsets of tuples, where each partition (group) has the same value for the grouping attributes.

IMPLEMENTING OUTER JOINS

- **Outer join** can be computed by modifying one of the join algorithms, such as nested-loop join or single-loop join.
- For **left outer join**, use the left relation as the outer loop or single-loop because every tuple in the left relation must appear in the result.
 - If there are **matching tuples** in the other relation, the joined tuples are produced and saved in the result.
 - If **no matching tuple** is found, the tuple is still included in the result but is padded with NULL value(s).
- The sort-merge and hash-join algorithms can also be extended to compute outer joins.

IMPLEMENTING OUTER JOINS

Example: `SELECT Lname, Fname, Dname FROM (EMPLOYEE LEFT OUTER JOIN DEPARTMENT ON Dno=Dnumber);`

- Compute the inner join of EMPLOYEE and DEPARTMENT tables.

$$T1 \leftarrow \Pi_{Lname, Fname, Dname}(EMPLOYEE \bowtie_{Dno=Dnumber} DEPARTMENT)$$

- Find the EMPLOYEE tuples that do not appear in inner JOIN result.

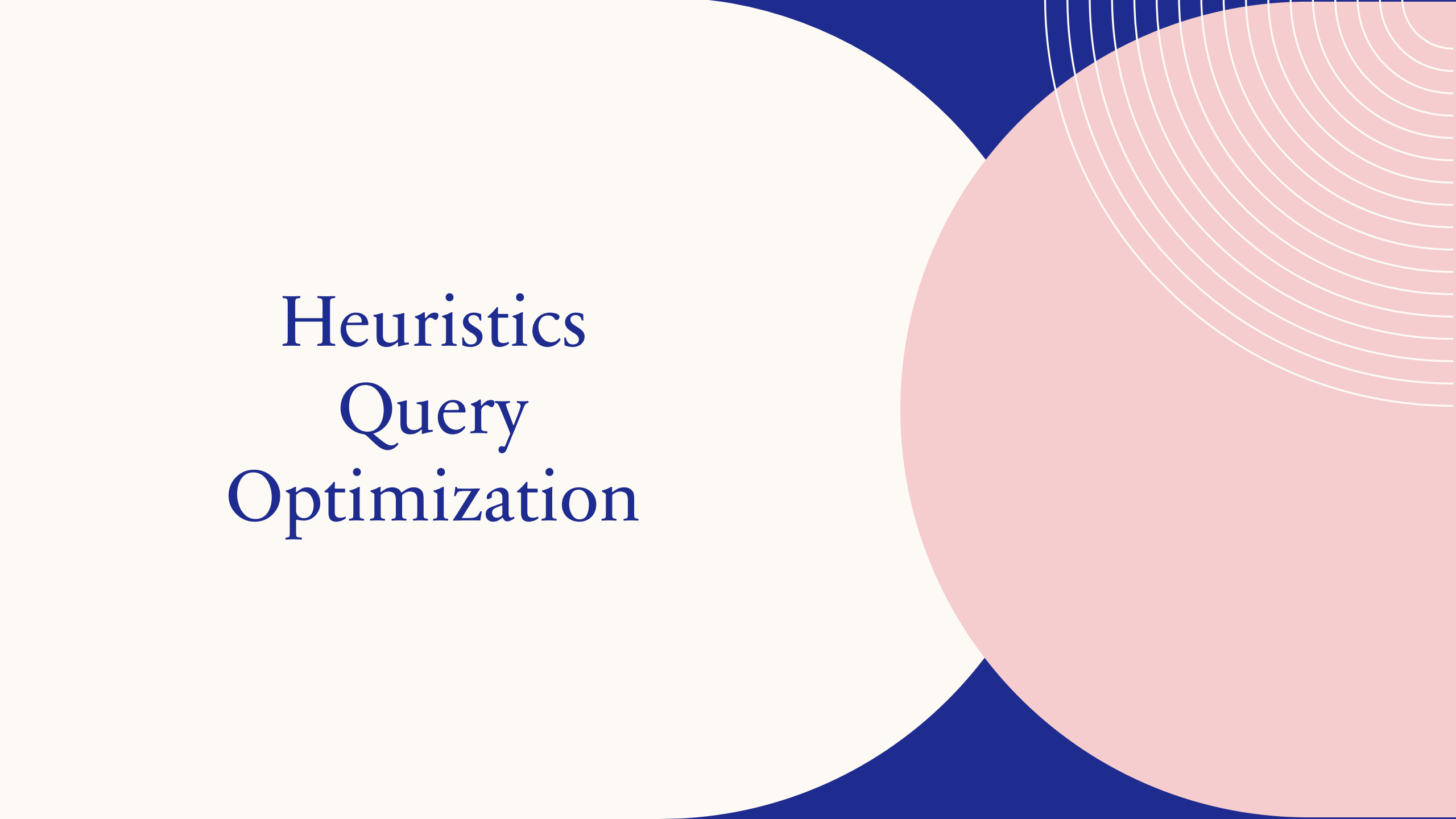
$$T2 \leftarrow \Pi_{Lname, Fname}(EMPLOYEE) - \Pi_{Lname, Fname}(T1)$$

- Pad each tuple in T2 with a NULL Dname field.

$$T2 \leftarrow T2 \times NULL$$

- Apply the UNION operation T1, T2 to produce the LEFT OUTER JOIN result.

$$RESULT \leftarrow T1 \cup T2$$

The background features a large white circle on the left and a large pink circle on the right, both partially overlapping a dark blue background. The pink circle contains several thin, white, concentric circular lines.

Heuristics Query Optimization

HEURISTICS IN QUERY OPTIMIZATION

- It is an optimization techniques that apply heuristic rules to modify the internal representation of a query.
- It is usually in the form of a query tree or a query graph data structure.
- It improves expected performance.
- The scanner and parser generate a data structure that corresponds to an initial query representation, which is then optimized according to heuristic rules.
- It produce an optimized query representation, which corresponds to the query execution strategy.
- Then, a query execution plan is generated to execute groups of operations based on access paths involved in the query.

HEURISTICS IN QUERY OPTIMIZATION

- **Heuristic Rule – Basic:**
- Apply SELECT and PROJECT operations before applying the JOIN or other binary operations, because the size of the file resulting from a JOIN, is usually a multiplicative function of the sizes of the input files.
- A **query tree** is used to represent a relational algebra or extended relational algebra expression, whereas a query graph is used to represent a relational calculus expression.

HEURISTICS IN QUERY OPTIMIZATION

- Heuristic Rule – Basic:
- Apply heuristic optimization rules are applied to convert an initial query tree into an **equivalent query tree**, which represents a different relational algebra expression that is more efficient to execute but gives the same result as the original tree.

NOTATION FOR QUERY TREES AND QUERY GRAPHS

- **Query tree:**
 - It is a tree data structure that corresponds to a relational algebra expression.
 - It represents the input relations of the query as **leaf nodes** of the tree, and represents the relational algebra operations as internal nodes.
- An execution of the query tree consists of executing an internal node operation whenever its operands are available and then replacing that internal node by the relation that results from executing the operation.

NOTATION FOR QUERY TREES AND QUERY GRAPHS

- The order of execution of operations *starts at the leaf nodes*, which represents the input database relations for the query, and *ends at the root node*, which represents the final operation of the query.
- The execution terminates when the root node operation is executed and produces the result relation for the query.

EMPLOYEE

Fname	Minit	Lname	<u>Ssn</u>	Bdate	Address	Sex	Salary	Super_ssn	Dno
-------	-------	-------	------------	-------	---------	-----	--------	-----------	-----

DEPARTMENT

Dname	<u>Dnumber</u>	Mgr_ssn	Mgr_start_date
-------	----------------	---------	----------------

DEPT_LOCATIONS

<u>Dnumber</u>	<u>Dlocation</u>
----------------	------------------

PROJECT

Pname	<u>Pnumber</u>	Plocation	Dnum
-------	----------------	-----------	------

WORKS_ON

<u>Essn</u>	<u>Pno</u>	Hours
-------------	------------	-------

DEPENDENT

<u>Essn</u>	<u>Dependent_name</u>	Sex	Bdate	Relationship
-------------	-----------------------	-----	-------	--------------

NOTATION FOR QUERY TREES AND QUERY GRAPHS

- For every project located in 'Stafford', retrieve the project number, the controlling department number, and the department manager's last name, address, and birthdate.

```
SELECT P.Pnumber, P.Dnum, E.Lname, E.Address, E.Bdate  
FROM PROJECT AS P, DEPARTMENT AS D, EMPLOYEE AS E  
WHERE P.Dnum=D.Dnumber AND D.Mgr_ssn=E.Ssn AND  
P.Plocation= 'Stafford';
```

- Relational Algebra:

$$\Pi_{Pnumber, Dnum, Lname, Address, Bdate}((\sigma_{Plocation="Stafford"}(Project))$$
$$\bowtie_{Dnum=Dnumber} (Department)) \bowtie_{Mgr-ssn=ssn} (EMPLOYEE))$$

NOTATION FOR QUERY TREES AND QUERY GRAPHS

- Relational Algebra:

$\Pi_{Pnumber,Dnum,Lname,Address,Bdate}((\sigma_{Plocation="Stafford"}(PROJECT) \bowtie_{Dnum=Dnumber} (DEPARTMENT)) \bowtie_{Mgr-ssn=ssn} (EMPLOYEE))$

- Query Block:

- Root: $\Pi_{Pnumber,Dnum,Lname,Address,Bdate}(Block3)$

- Block 3: $Block2 \bowtie_{Mgr-ssn=ssn} (EMPLOYEE)$

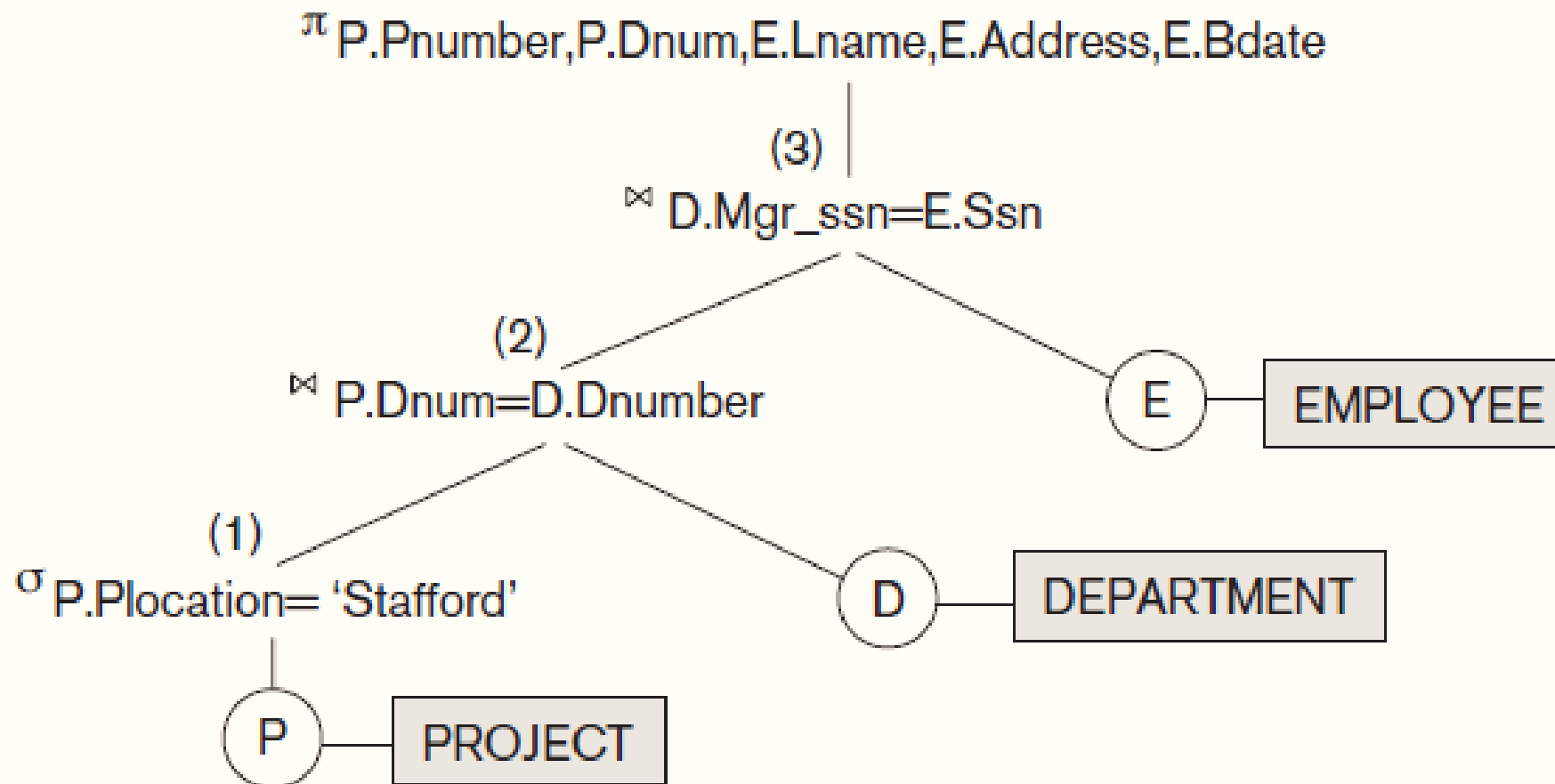
- Block 2: $Block1 \bowtie_{Dnum=Dnumber} (DEPARTMENT)$

- Block 1: $\sigma_{Plocation="Stafford"}(PROJECT)$

- Leaf node P, D, and E represent three relations PROJECT, DEPARTMENT & EMPLOYEE.

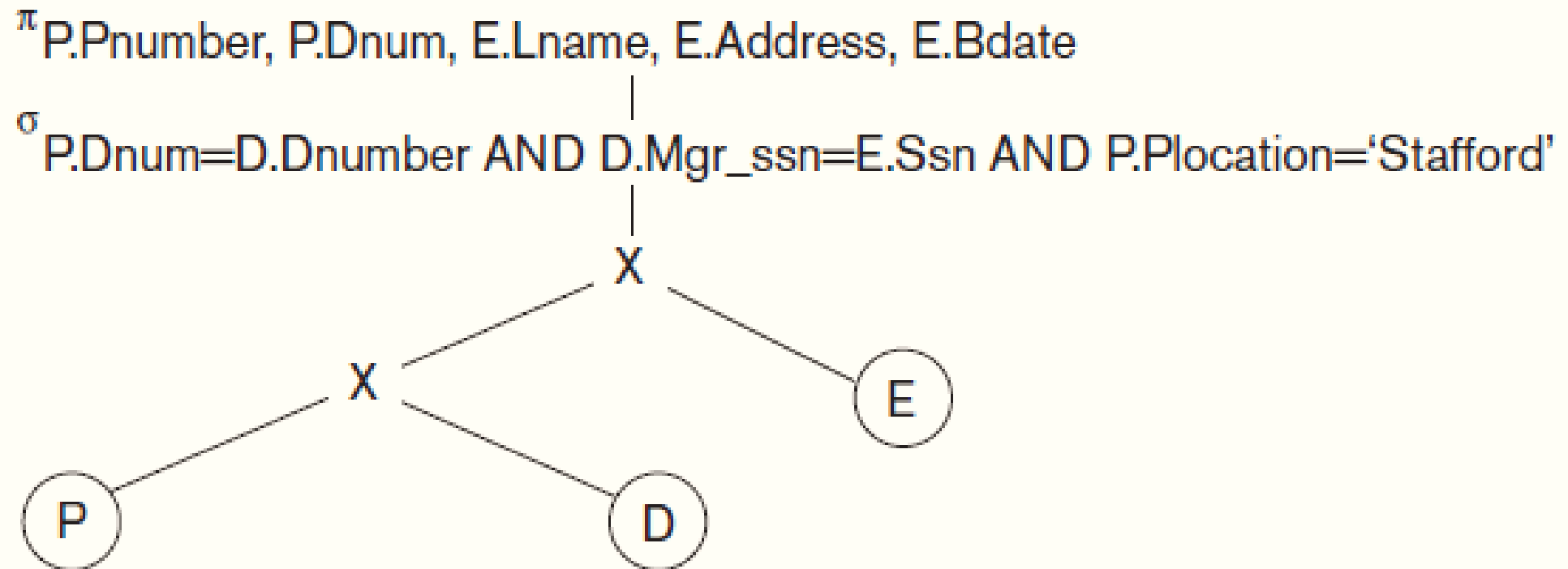
NOTATION FOR QUERY TREES AND QUERY GRAPHS

- Query tree:



NOTATION FOR QUERY TREES AND QUERY GRAPHS

- Initial (canonical) Query tree:

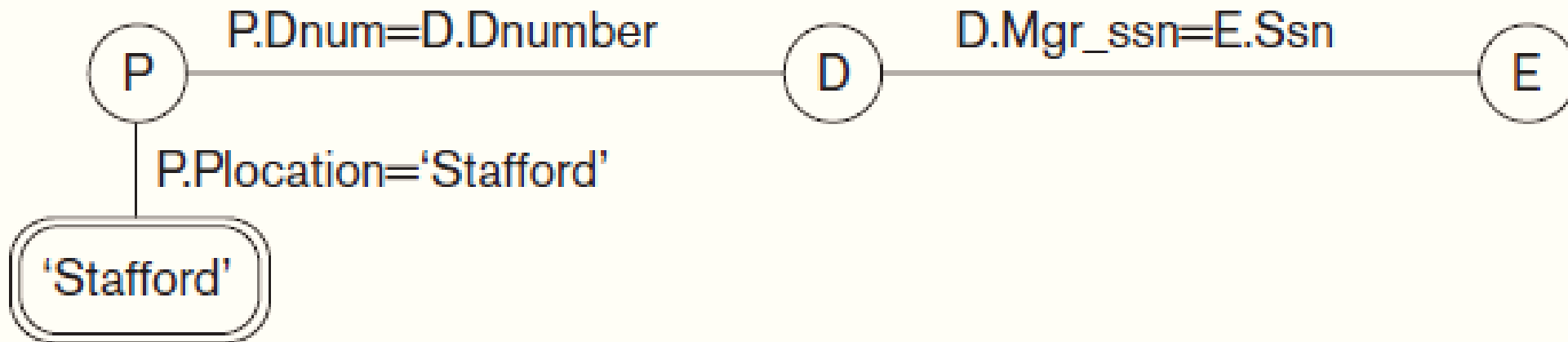


NOTATION FOR QUERY TREES AND QUERY GRAPHS

- **Query Graph:** A neutral data structure for representing query tree in a specific order of operations for execution a query.
- Query Graph for query:

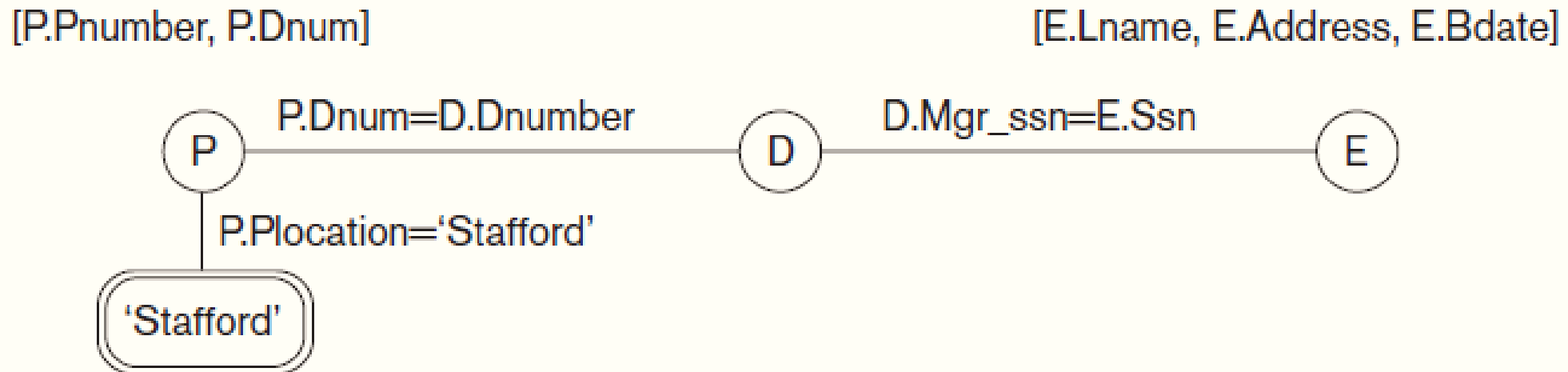
[P.Pnumber, P.Dnum]

[E.Lname, E.Address, E.Bdate]



NOTATION FOR QUERY TREES AND QUERY GRAPHS

- **Relation nodes:** Relations in the query, which are displayed as single circles.
- **Constant nodes:** - Constant values, from the query selection conditions, which are displayed as double circles or ovals.
- **Graph Edges** – represent the Selection and join conditions.

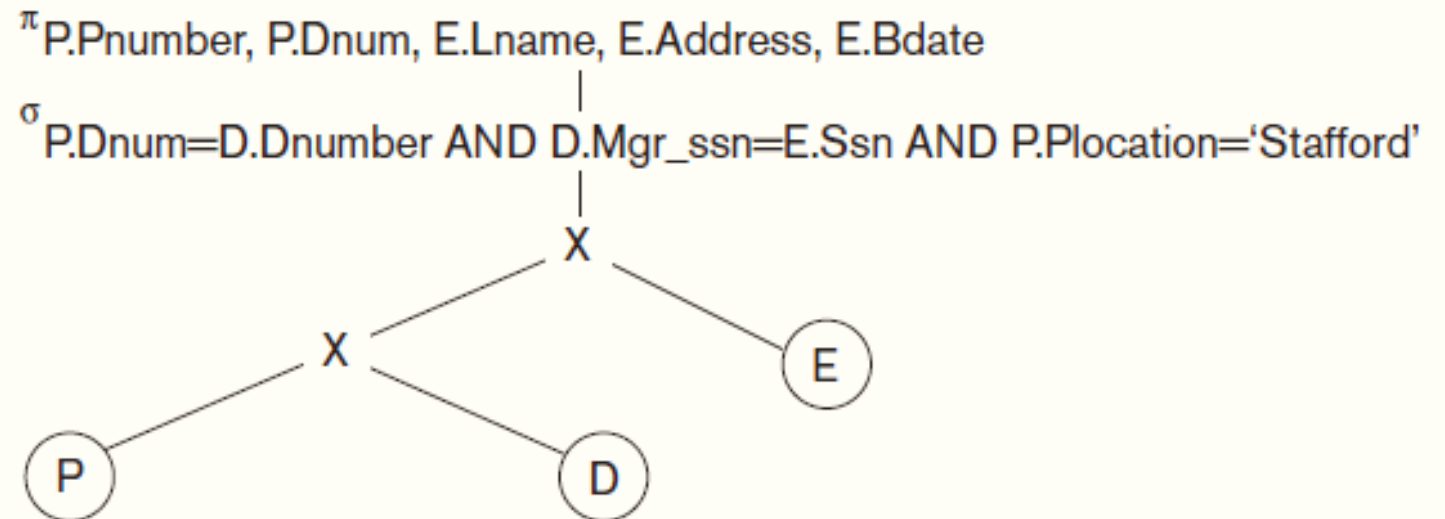


NOTATION FOR QUERY TREES AND QUERY GRAPHS

- The query graph representation **does not indicate an order on which operations to perform first.**
- There is **only a single graph corresponding to each query.**
- Query optimizer needs to show the order of operations for query execution, which is not possible in query graphs.
- Different relational algebra expressions, and hence many different query trees, can be **equivalent**; that is, they can represent the same query.

HEURISTIC OPTIMIZATION OF QUERY TREES

- The query parser will typically generate a standard initial query tree to correspond to an SQL query, without doing any optimization.
- The CARTESIAN PRODUCT of the relations specified in the FROM clause is first applied; then the selection and join conditions of the WHERE clause are applied, followed by the projection on the SELECT clause attributes.



HEURISTIC OPTIMIZATION OF QUERY TREES

- Such a canonical query tree represents a relational algebra expression that is **very inefficient** if executed directly, because of the CARTESIAN PRODUCT ($\times$) operations.
- Example:
 - PROJECT – 100 bytes of 100 tuples
 - DEPARTMENT – 50 bytes of 20 tuples
 - EMPLOYEE – 150 bytes of 5000 tuples.
 - CARTESIAN PRODUCT = $100 \times 20 \times 5000 = 10000000$ tuples of record of size 300 bytes (100+50+150).

HEURISTIC OPTIMIZATION OF QUERY TREES

- **Initial query tree** is in a simple standard form that can be easily created from the SQL query.
- It **will never be executed**.
- The heuristic query optimizer will transform this initial query tree into an equivalent **final query tree** that is efficient to execute.
- The optimizer must include
 - Rules for equivalence among relational algebra expressions that can be applied to transform the initial tree into the final,
 - Optimized query tree.

HEURISTIC OPTIMIZATION OF QUERY TREES

- **Transforming a Query:**
- If two relations have the same set of attributes in a different order but the two relations represent the same information, we consider the relations to be equivalent.
- **Transformation rules** that are useful in query optimization, without proving them:

HEURISTIC OPTIMIZATION OF QUERY TREES

- **Transformation Rule:**
- Cascade of σ
- Commutativity of σ
- Cascade of Π
- Commuting σ with Π
- Commutating of $\bowtie$
- Commuting σ with $\bowtie$
- Commuting Π with $\bowtie$
- Commutativity of SET OPERATIONS $\cup, \cap, -$
- Associativity of $\bowtie, \times, \cup, \cap$
- Commuting σ with SET OPERATIONS
- Π commutes with $\cup$
- Converting $(\sigma, \times)$ sequence into $\bowtie$

HEURISTIC OPTIMIZATION OF QUERY TREES

- **Transformation Rule:**

- Cascade of σ :

$$\sigma_{c1 \text{ AND } c2 \text{ AND } \dots \text{ AND } cn}(R) \equiv \sigma_{c1}(\sigma_{c2}(\dots(\sigma_{cn}(R))\dots))$$

- Commutativity of σ :

$$\sigma_{c1}(\sigma_{c2}(R)) \equiv \sigma_{c2}(\sigma_{c1}(R))$$

- Cascade of Π :

$$\Pi_{List1}(\Pi_{List2}(\dots(\Pi_{Listn}(R))\dots)) \equiv \Pi_{List1}(R)$$

- Commuting σ with Π :

$$\Pi_{A1,A2,\dots,An}(\sigma_c(R)) \equiv \sigma_c(\Pi_{A1,A2,\dots,An}(R))$$

HEURISTIC OPTIMIZATION OF QUERY TREES

- Transformation Rule:
- Commutating of $\bowtie$:

$$R \bowtie_c S \equiv S \bowtie_c R$$
$$R \times S \equiv S \times R$$

- Commuting σ with $\bowtie$:

$$\sigma_c(R \bowtie S) \equiv (\sigma_c(R)) \bowtie S$$
$$\sigma_c(R \bowtie S) \equiv (\sigma_{c1}(R)) \bowtie (\sigma_{c2}(R))$$

- Commuting Π with $\bowtie$:

$$\Pi_L(R \bowtie_c S) \equiv \left(\Pi_{A1, \dots, An}(R) \right) \bowtie_c \left(\Pi_{B1, \dots, Bn}(S) \right)$$
$$\Pi_L(R \bowtie_c S) \equiv \Pi_L((\Pi_{A1, \dots, An, An+1, \dots, An+k}(R)) \bowtie_c (\Pi_{B1, \dots, Bm, Bm+1, \dots, Bm+p}(S)))$$

HEURISTIC OPTIMIZATION OF QUERY TREES

- Commutativity of SET OPERATIONS:

$\cup, \cap$, are commutative, but $-$ is not.

- Associativity of $\bowtie, \times, \cup, \cap$:

Let θ be any set operation then $(R \theta S) \theta T \equiv R \theta (S \theta T)$

- Commuting σ with SET OPERATIONS:

Let θ be any set operation then $\sigma_c(R \theta S) \equiv (\sigma_c(R)) \theta (\sigma_c(S))$

- Π commutes with $\cup$:

$$\Pi_L(R \cup S) \equiv (\Pi_L(R)) \cup (\Pi_L(S))$$

- Converting $(\sigma, \times)$ sequence into $\bowtie$:

$$(\sigma_c(R \times S)) \equiv (R \bowtie_c S)$$

EMPLOYEE

Fname	Minit	Lname	<u>Ssn</u>	Bdate	Address	Sex	Salary	Super_ssn	Dno
-------	-------	-------	------------	-------	---------	-----	--------	-----------	-----

DEPARTMENT

Dname	<u>Dnumber</u>	Mgr_ssn	Mgr_start_date
-------	----------------	---------	----------------

DEPT_LOCATIONS

<u>Dnumber</u>	<u>Dlocation</u>
----------------	------------------

PROJECT

Pname	<u>Pnumber</u>	Plocation	Dnum
-------	----------------	-----------	------

WORKS_ON

<u>Essn</u>	<u>Pno</u>	Hours
-------------	------------	-------

DEPENDENT

<u>Essn</u>	<u>Dependent_name</u>	Sex	Bdate	Relationship
-------------	-----------------------	-----	-------	--------------

EXAMPLE FOR HEURISTIC OPTIMIZATION

- Find the last names of employees born after 1957 who work on a project named “Aquarius”

```
SELECT Lname
FROM EMPLOYEE, WORKS_ON, PROJECT
WHERE Pname='Aquarius' AND Pnumber=Pno AND Essn=Ssn
AND Bdate > '1957-12-31';
```

- RA Query:

$$\Pi_{Lname}(\sigma_{Pname='Aquarius' \wedge Pnumber=Pno \wedge Essn=ssn \wedge Bdate > "1957-12-31"}(EMPLOYEE \times WORKSON \times PROJECT))$$

EXAMPLE FOR HEURISTIC OPTIMIZATION

- Query Block:

$$\Pi_{Lname}(Block)$$

- Block 1:

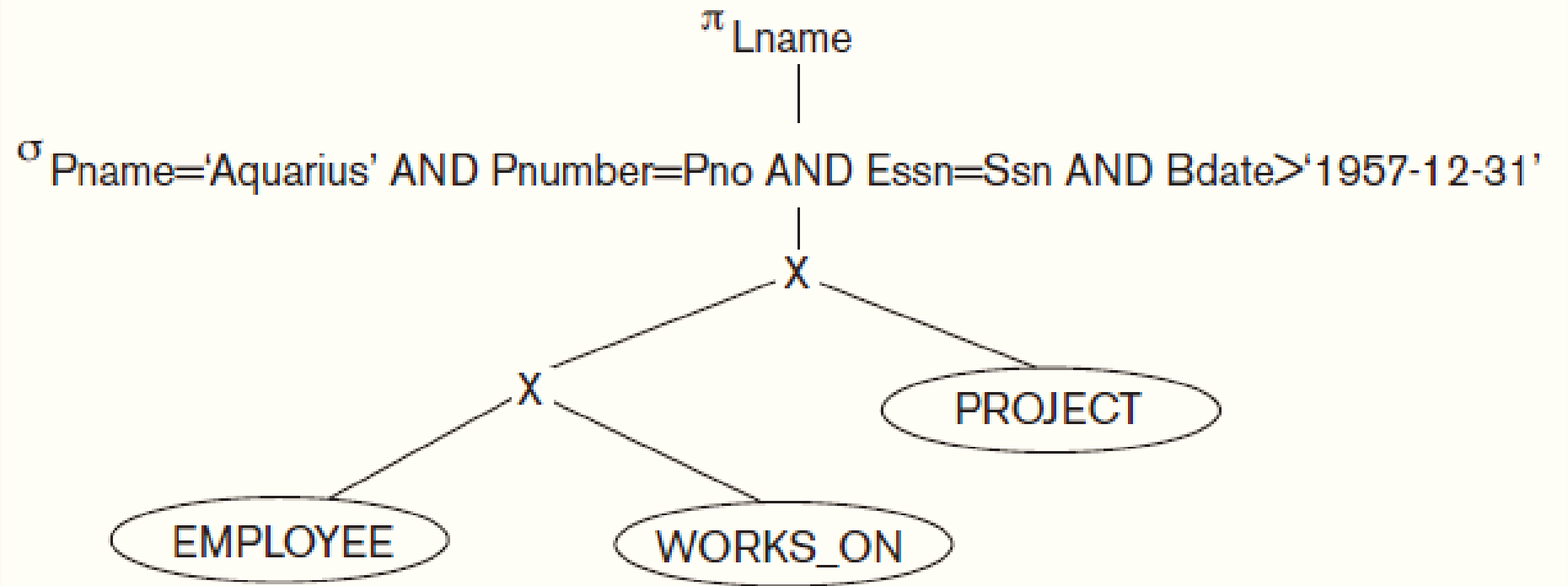
$$\sigma_{Pname='Aquarius' \wedge Pnumber=Pno \wedge Essn=ssn \wedge Bdate > "1957-12-31"}(Block2)$$

- Block 2:

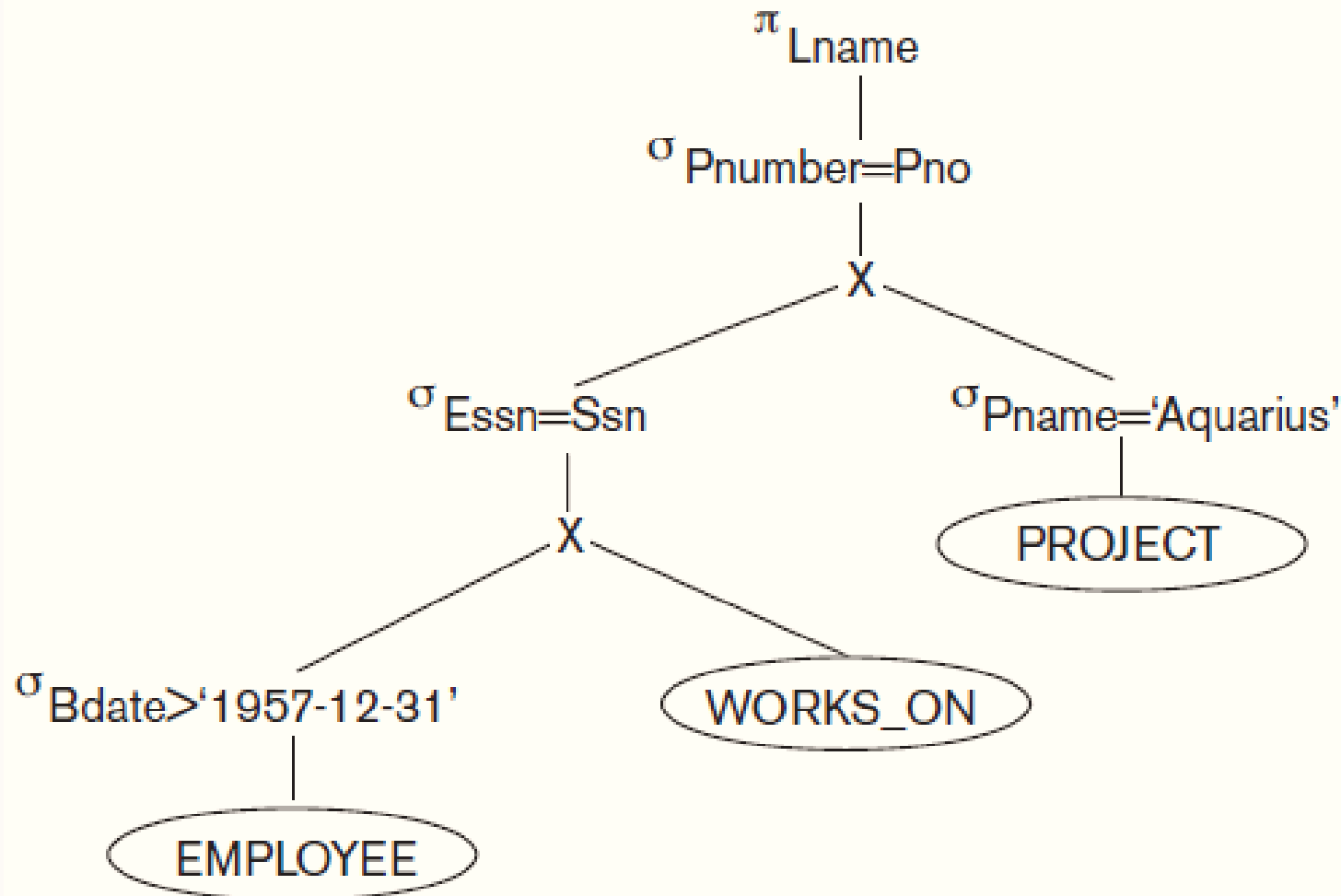
$$(EMPLOYEE \times WORKSON \times PROJECT)$$

EXAMPLE FOR HEURISTIC OPTIMIZATION

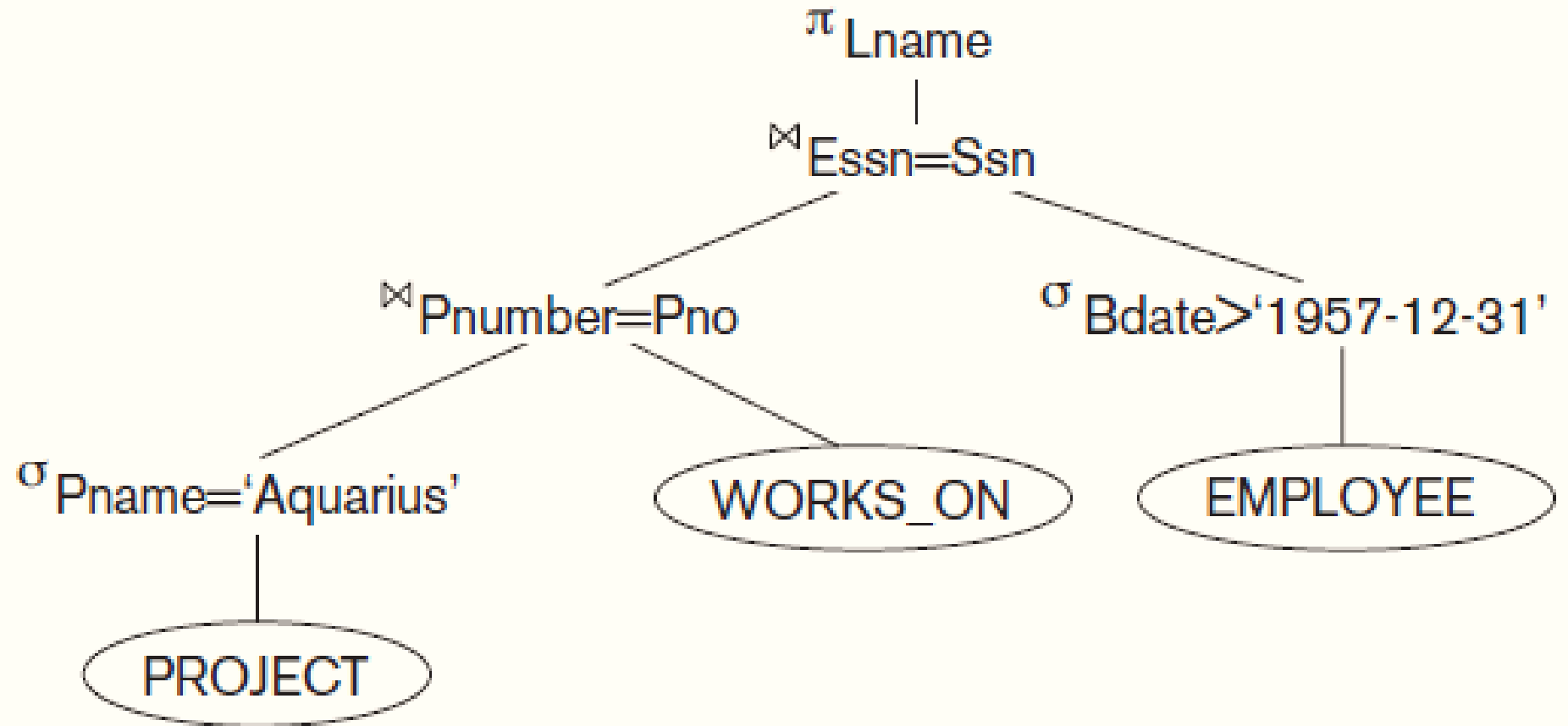
- Query Tree:



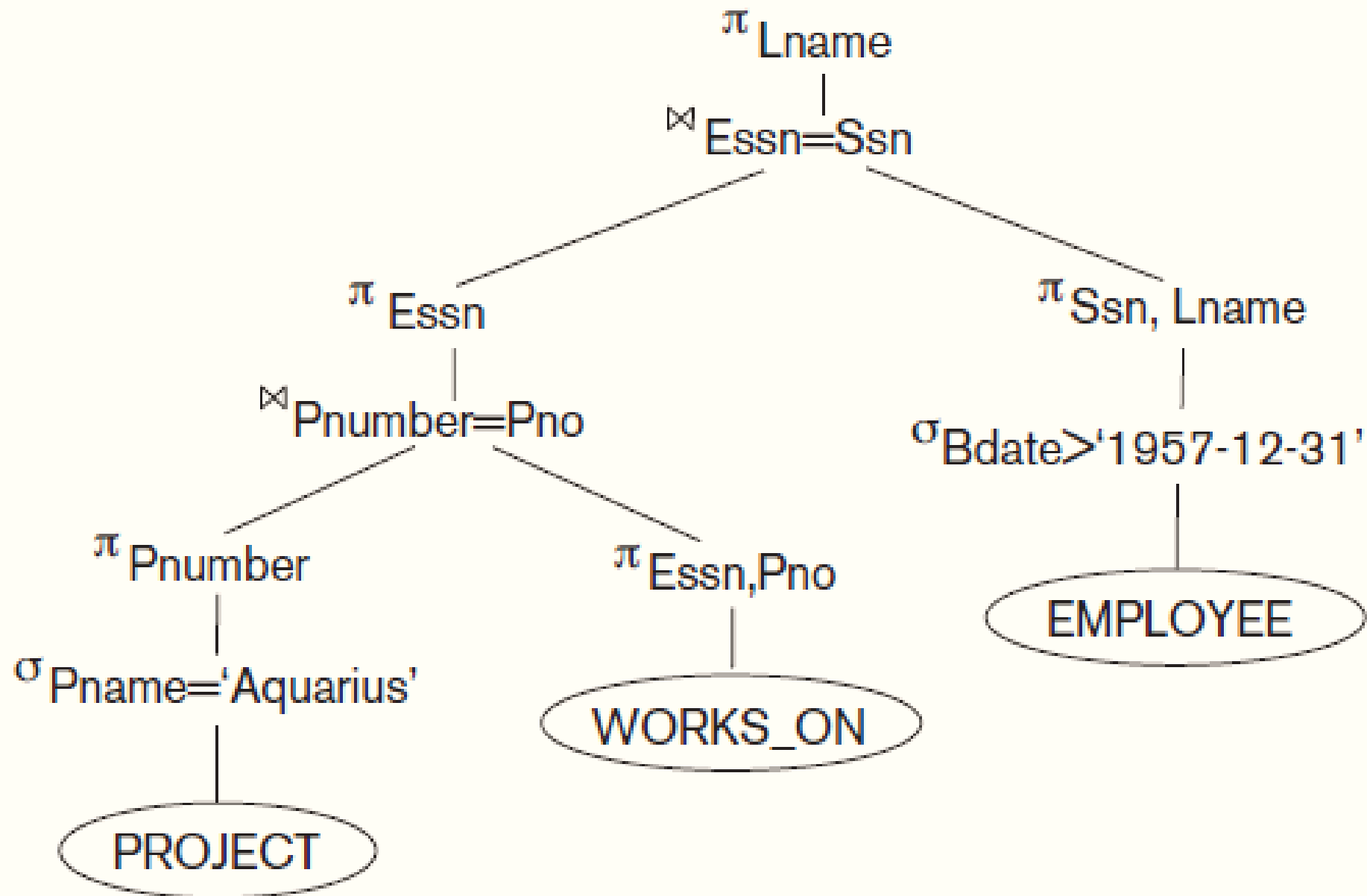
- **Moving SELECT operation down the Query tree:**
- Using Rule 1, break up any SELECT operations with conjunctive conditions into a cascade of SELECT operations.



- Applying the more restrictive SELECT operation first:
- Using Rules 2, 4, 6, and 10 concerning the commutativity of SELECT with other operations, move each SELECT operation as far down the query tree as is permitted by the attributes involved in the select condition.



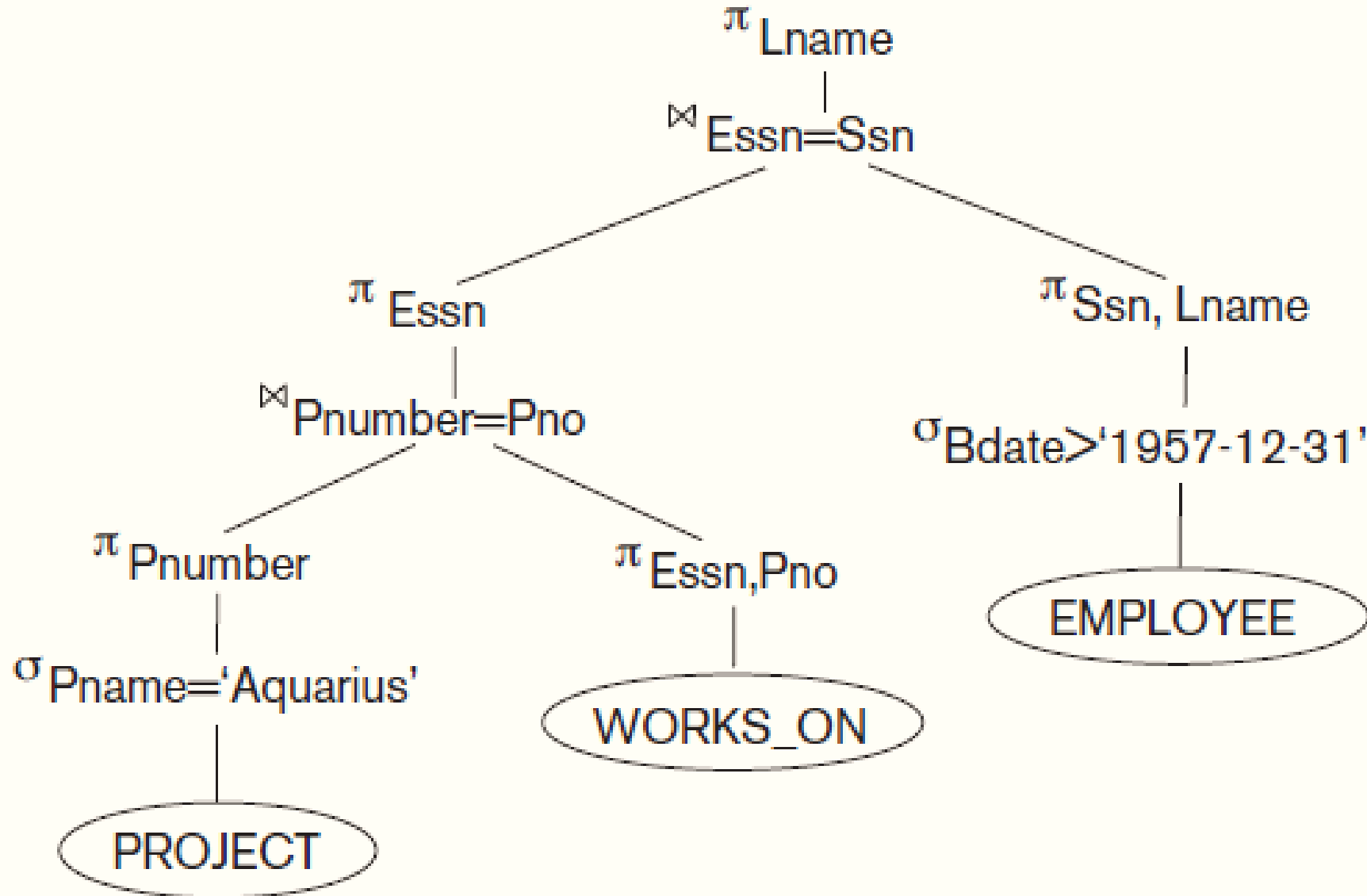
- Replacing CARTESIAN PRODUCT and SELECT with JOIN:



- **Replacing CARTESIAN PRODUCT and SELECT with JOIN:**
- Using Rules 5 and 9 concerning commutativity and associativity of binary operations, rearrange the leaf nodes of the tree using the following criteria.
- First, position the leaf node relations with the most restrictive SELECT operations so they are executed first in the query tree representation.
- The definition of most restrictive SELECT can mean either the ones that produce a relation with the fewest tuples or with the smallest absolute size.
- Another possibility is to define the most restrictive SELECT as the one with the smallest selectivity; this is more practical because estimates of selectivities are often available in the DBMS catalog.

- 
- **Replacing CARTESIAN PRODUCT and SELECT with JOIN:**
 - Second, make sure that the ordering of leaf nodes does not cause CARTESIAN PRODUCT operations; for example, if the two relations with the most restrictive SELECT do not have a direct join condition between them, it may be desirable to change the order of leaf nodes to avoid Cartesian products.
 - Using Rule 12, combine a CARTESIAN PRODUCT operation with a subsequent SELECT operation in the tree into a JOIN operation, if the condition represents a join condition.

- Moving PROJECT operations down the Query tree:



- 
- **Moving PROJECT operations down the Query tree :**
 - Using Rules 3, 4, 7, and 11 concerning the cascading of PROJECT and the commuting of PROJECT with other operations, break down and move lists of projection attributes down the tree as far as possible by creating new PROJECT operations as needed.
 - Only those attributes needed in the query result and in subsequent operations in the query tree should be kept after each PROJECT operation.
 - Identify subtrees that represent groups of operations that can be executed by a single algorithm.

QUERYING EXECUTION PLANS

- An execution plan for a relational algebra expression represented as a query tree includes information about the access methods available for each relation as well as the algorithms to be used in computing the relational operators represented in the tree.
- Two Type Evaluation:
 - **Materialized evaluation:** result of an operation is stored as a temporary relation (that is, the result is physically materialized).
 - **Pipelined evaluation:** resulting tuples of an operation are produced, they are forwarded directly to the next operation in the query sequence.
- The advantage of pipelining is the cost savings in not having to write the intermediate results to disk and not having to read them back for the next operation.

QUERYING EXECUTION PLANS

- **Example:**

$$\Pi_{Fname,Lname,Address} (\sigma_{Dname="Research"}(DEPARTMENT) \bowtie_{Dnumber=Dno} EMPLOYEE)$$

- To convert this into an execution plan, optimizer choose
 - an index search for the SELECT operation on DEPARTMENT
 - a single-loop join algorithm over the result of the SELECT operation on DEPARTMENT for the join operation (index exists on the Dno attribute of EMPLOYEE)
 - a scan of the JOIN result for input to the PROJECT operator.

COST ESTIMATES IN QUERY OPTIMIZATION

- A query optimizer does not depend solely on heuristic rules; it also estimates and compares the costs of executing a query using different execution strategies and algorithms, and it then chooses the strategy with the *lowest cost estimate*.
- For this approach to work, accurate **cost estimates** are required so that different strategies can be compared fairly and realistically.
- The optimizer must limit the number of execution strategies to be considered; otherwise, too much time will be spent making cost estimates for the many possible execution strategies.

COST ESTIMATES IN QUERY OPTIMIZATION

- Cost Estimate Approach uses
 - **Compiled queries** - where the optimization is done at compile time and the resulting execution strategy code is stored and executed directly at runtime.
 - **Interpreted queries** - where the entire process occurs at runtime, a full-scale optimization may slow down the response time.
- **Cost based Query Optimization:**
 - It uses traditional optimization techniques that search the solution space to a problem for a solution that minimizes an objective (cost) function.
- The cost functions used in query optimization are estimates and not exact cost functions, so the optimization may select a query execution strategy that is not the optimal (absolute best) one.

COST COMPONENT FOR QUERY OPTIMIZATION

- **Access cost to secondary Storage**
 - This is the cost of transferring (reading and writing) data blocks between secondary disk storage and main memory buffers.
 - The cost of searching for records in a disk file depends on the type of access structures on that file, such as ordering, hashing, and primary or secondary indexes.
 - This is also known as I/O cost.
- **Disk storage cost**
 - The cost of storing on disk any intermediate files that are generated by an execution strategy for the query

COST COMPONENT FOR QUERY OPTIMIZATION

- **Computation cost**
 - The cost of performing in-memory operations on the records within the data buffers during query execution.
 - Operations include searching for and sorting records, merging records for a join or a sort operation, and performing computations on field values.
 - This is also known as **CPU (central processing unit) cost**.
- **Memory usage cost**
 - The cost pertaining to the number of main memory buffers needed during query execution.
- **Communication cost**
 - This is the cost of shipping the query and its results from the database site to the site or terminal where the query originated.

COST FUNCTIONS FOR SELECT

- **S1—Linear search (brute force) approach.**
 - File blocks to retrieve all records satisfying the selection condition. Cost $C_{S1a} = b$.
 - For equality condition on a key attribute, only half the file blocks are searched on the average before finding the record, so a estimate for $C_{S1b} = (b/2)$
 - If the record is found; if no record is found then estimate for $C_{S1b} = b$
- **S2—Binary search.**
 - This search accesses approximately $CS2 = \log_2 b + \lceil (s/bfr) \rceil - 1$ file blocks.

COST FUNCTIONS FOR SELECT

- **S3a—Using a primary index to retrieve a single record.**
 - For a primary index, retrieve one disk block at each index level, plus one disk block from the data file.
 - The cost is one more disk block than the number of index levels: $C_{S3a} = x + 1$.
- **S3b—Using a hash key to retrieve a single record.**
 - The cost function is approximately $C_{S3b} = 1$ for static hashing or linear hashing, and it is 2 disk block accesses for extendible hashing.
- **S4—Using an ordering index to retrieve multiple records.**
 - If the comparison condition is $>$, $\geq$, $<$, or $\leq$ on a key field with an ordering index, roughly half the file records will satisfy the condition.
 - Cost function of $C_{S4} = x + (b/2)$.

COST FUNCTIONS FOR SELECT

- **S5—Using a clustering index to retrieve multiple records.**
 - One disk block is accessed at each index level, which gives the address of the first file disk block in the cluster.
 - For $\lceil (s/bfr) \rceil$ file blocks will be in the cluster of file blocks that hold all the selected records, giving $C_{S5} = x + \lceil (s/bfr) \rceil$.
- **S6—Using a secondary (B+-tree) index.**
 - Secondary index on a key (unique) attribute, the cost is $x + 1$ disk block accesses.
 - Index is nonclustering, each of the records may reside on a different disk block, so the (worst case) cost estimate is $C_{S6a} = x + 1 + s$.
 - If the comparison condition is $>$, $>=$, $<$, or $<=$ and half the file records are assumed to satisfy the condition, then (very roughly) half the first-level index blocks are accessed, plus half the file records via the index.
 - The cost estimate for this case, approximately, is $C_{S6b} = x + (bI/2) + (r/2)$.

COST FUNCTIONS FOR SELECT

- **S7—Conjunctive selection.**
 - Use either S1 or one of the methods S2 to S6 discussed above.
- **S8—Conjunctive selection using a composite index.**
 - Same as S3a, S5, or S6a, depending on the type of index.

COST FUNCTIONS FOR SELECT

- Suppose that the EMPLOYEE file in Figure 3.5 has $r_E = 10,000$ records
- stored in $b_E = 2000$ disk blocks with blocking factor $bfr_E = 5$ records/block and the following access paths:
- A clustering index on Salary, with levels $x_{Salary} = 3$ and average selection cardinality $s_{Salary} = 20$. (This corresponds to a selectivity of $sl_{Salary} = 0.002$).
- A secondary index on the key attribute Ssn, with $x_{Ssn} = 4$ ($s_{Ssn} = 1$, $sl_{Ssn} = 0.0001$).
- A secondary index on the nonkey attribute Dno, with $x_{Dno} = 2$ and first-level index blocks $b_{I1Dno} = 4$. There are $d_{Dno} = 125$ distinct values for Dno, so the selectivity of Dno is $sl_{Dno} = (1/d_{Dno}) = 0.008$, and the selection cardinality is $s_{Dno} = (r_E * sl_{Dno}) = (r_E/d_{Dno}) = 80$.

COST FUNCTIONS FOR SELECT

- Suppose that the EMPLOYEE file in Figure 3.5 has $r_E = 10,000$ records
- stored in $b_E = 2000$ disk blocks with blocking factor $bfr_E = 5$ records/block and the following access paths:
- A secondary index on Sex, with $x_{Sex} = 1$. There are $d_{Sex} = 2$ values for the Sex attribute, so the average selection cardinality is $s_{Sex} = (r_E/d_{Sex}) = 5000$.

COST FUNCTIONS FOR SELECT

- Illustrate the use of cost functions with the following examples:
- OP1: $\sigma_{Ssn='123456789'}(EMPLOYEE)$
- OP2: $\sigma_{Dno>5}(EMPLOYEE)$
- OP3: $\sigma_{Dno=5}(EMPLOYEE)$
- OP4: $\sigma_{Dno=5 \text{ AND } SALARY>30000 \text{ AND } Sex='F'}(EMPLOYEE)$
- The cost of the brute force using option S1 as $C_{S1a} = b_E = 2000$ (for a selection on a nonkey attribute)
- $C_{S1b} = (b_E/2) = 1000$ (average cost for a selection on a key attribute).

COST FUNCTIONS FOR SELECT

- Illustrate the use of cost functions with the following examples:
- OP1: $\sigma_{Ssn='123456789'}(EMPLOYEE)$
- OP2: $\sigma_{Dno>5}(EMPLOYEE)$
- OP3: $\sigma_{Dno=5}(EMPLOYEE)$
- OP4: $\sigma_{Dno=5 \text{ AND } SALARY>30000 \text{ AND } Sex='F'}(EMPLOYEE)$
- For OP1:
- Use S1 or method S6a
- Cost estimate for S6a is $C_{S6a} = x_{Ssn} + 1 = 4 + 1 = 5$,
- Cost estimate for S1 is $C_{S1b} = 1000$.

COST FUNCTIONS FOR SELECT

- Illustrate the use of cost functions with the following examples:
- OP2: $\sigma_{Dno>5}(EMPLOYEE)$
- For OP2:
- Use either method S1 (with estimated cost $C_{S1a} = 2000$) or
- S6b (with estimated cost $C_{S6b} = x_{Dno} + (b_{I1Dno}/2) + (r_E /2) = 2 + (4/2) + (10,000/2) = 5004$)

COST FUNCTIONS FOR SELECT

- Illustrate the use of cost functions with the following examples:
- OP3: $\sigma_{Dno=5}(EMPLOYEE)$
- For OP3:
- Use either method S1 (with estimated cost $C_{S1a} = 2000$) or
- method S6a (with estimated cost $C_{S6a} = x_{Dno} + s_{Dno} = 2 + 80 = 82$), so we choose method S6a.

COST FUNCTIONS FOR SELECT

- Illustrate the use of cost functions with the following examples:
- OP4: $\sigma_{Dno=5 \text{ AND } SALARY>30000 \text{ AND } Sex='F'}(EMPLOYEE)$
- For OP3:
- Has a conjunctive selection condition.
- Cost Estimate using any one of the three components of the selection condition to retrieve the records, plus the linear search approach.
- The latter gives cost estimate $C_{S1a} = 2000$.
- Using the condition $(Dno = 5)$ first gives the cost estimate $C_{S6a} = 82$.
- Using the condition $(Salary > 30,000)$ first gives a cost estimate $C_{S4} = x_{Salary} + (b_E/2) = 3 + (2000/2) = 1003$.
- Using the condition $(Sex = 'F')$ first gives a cost estimate $C_{S6a} = x_{Sex} + s_{Sex} = 1 + 5000 = 5001$.

COST FUNCTIONS FOR JOIN

- The join operations are of the form: $R \bowtie_{A=B} S$ where **A** and **B** are domain-compatible attributes of **R** and **S**, respectively.
- Assume that **R** has b_R blocks and that **S** has b_S blocks:
- **J1—Nested-loop join.**
 - Suppose that we use **R** for the outer loop; Cost function to estimate the number of block accesses for this method, assuming **three memory buffers**.
 - Blocking factor for file is bfr_{RS} and that the join selectivity **js** is known: $C_{J1} = b_R + (b_R * b_S) + ((js * |R| * |S|)/bfr_{RS})$
 - If n_B main memory buffers are available to perform the join, the cost formula becomes: $C_{J1} = b_R + (\lceil b_R / (n_B - 2) \rceil * b_S) + ((js * |R| * |S|)/bfr_{RS})$

COST FUNCTIONS FOR JOIN

- **J2—Single-loop join (using an access structure to retrieve the matching record(s))**
 - For a secondary index where s_B is the selection cardinality for the join attribute B of S21 we get: $C_{J2a} = b_R + (|R| * (x_B + 1 + s_B)) + ((j_s * |R| * |S|)/bfr_{RS})$
 - For a clustering index where s_B is the selection cardinality of B, we get $C_{J2b} = b_R + (|R| * (x_B + (s_B/bfr_B))) + ((j_s * |R| * |S|)/bfr_{RS})$
 - For a primary index, we get $C_{J2c} = b_R + (|R| * (x_B + 1)) + ((j_s * |R| * |S|)/bfr_{RS})$
 - If a hash key exists for one of the two join attributes—say, B of S—we get $C_{J2d} = b_R + (|R| * h) + ((j_s * |R| * |S|)/bfr_{RS})$

COST FUNCTIONS FOR JOIN

- **J3—Sort-merge join.**
 - If the files are already sorted on the join attributes, the cost function for this method is $C_{J3a} = b_R + b_S + ((js * |R| * |S|)/bfr_{RS})$
- Example:
- EMPLOYEE $r_E = 10000$ in records $b_E = 2000$ blocks and DEPARTMENT file: $r_D = 125$ in $b_D = 13$ disk blocks.
- Consider the following two join operations:
- OP6: EMPLOYEE $\bowtie_{Dno=Dnumber}$ DEPARTMENT
- OP7: DEPARTMENT $\bowtie_{Mgr_ssn=Ssn}$ EMPLOYEE

COST FUNCTIONS FOR JOIN

- Example: Consider the following two join operations:
- OP6: $\text{EMPLOYEE} \bowtie_{\text{Dno=Dnumber}} \text{DEPARTMENT}$
- OP7: $\text{DEPARTMENT} \bowtie_{\text{Mgr_ssn=Ssn}} \text{EMPLOYEE}$
- EMPLOYEE $r_E = 10000$ in records $b_E = 2000$ blocks and DEPARTMENT file: $r_D = 125$ in $b_D = 13$ disk blocks.
- Suppose a primary index on Dnumber of DEPARTMENT with $x_{\text{Dnumber}} = 1$ level and a secondary index on Mgr_ssn of DEPARTMENT with selection cardinality $s_{\text{Mgr_ssn}} = 1$ and levels $x_{\text{Mgr_ssn}} = 2$.
- Assume that the join selectivity for OP6 is $j_{s_{\text{OP6}}} = (1/|\text{DEPARTMENT}|) = 1/125$ because Dnumber is a key of DEPARTMENT.
- $bfr_{ED} = 4$ records per block

COST FUNCTIONS FOR JOIN

- Worst-case costs for the JOIN operation OP6 using methods J1 and J2 as follows:
- Using method J1 with EMPLOYEE as outer loop:
 - $C_{J1} = b_E + (b_E * b_D) + ((js_{OP6} * r_E * r_D)/bfr_{ED}) = 2000 + (2000 * 13) + (((1/125) * 10,000 * 125)/4) = 30,500$
- Using method J1 with DEPARTMENT as outer loop:
 - $C_{J1} = b_D + (b_E * b_D) + ((js_{OP6} * r_E * r_D)/bfr_{ED}) = 13 + (13 * 2000) + (((1/125) * 10,000 * 125)/4) = 28,513$

COST FUNCTIONS FOR JOIN

- Worst-case costs for the JOIN operation OP6 using methods J1 and J2 as follows:
- Using method J2 with EMPLOYEE as outer loop:
 - $C_{J2c} = b_E + (r_E * (x_{D_{number}} + 1)) + ((js_{OP6} * r_E * r_D) / bfr_{ED}) = 2000 + (10,000 * 2) + (((1/125) * 10,000 * 125/4) = 24,500$
- Using method J2 with DEPARTMENT as outer loop:
 - $C_{J2a} = b_D + (r_D * (x_{D_{no}} + s_{D_{no}})) + ((js_{OP6} * r_E * r_D) / bfr_{ED}) = 13 + (125 * (2 + 80)) + (((1/125) * 10,000 * 125/4) = 12,763$



THANK YOU

Module 4

BCSE302L – Database Systems